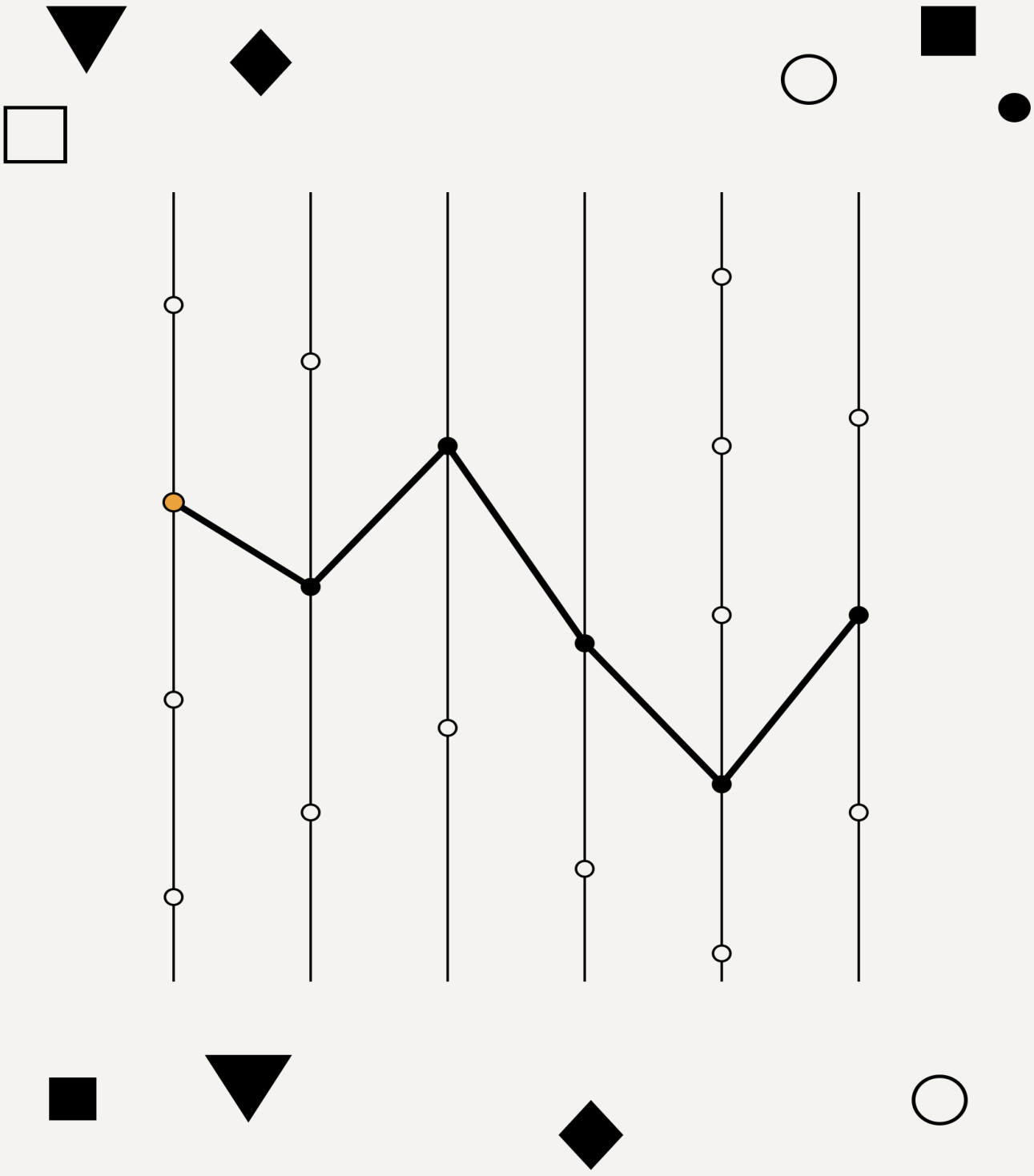
Architecture Synthesis

The Next Correct Step



Sergiy Yevtushenko

*

Architecture Synthesis

The Next Correct Step

Sergiy Yevtushenko

Version 1.0.1 (Draft) — 29 July 2026

Contents

About the Series

This book is one layer of a series that attempts something specific: deriving software systems from business intent through progressively smaller decision spaces.

> Business intent → **Process-First Design** (how the application thinks) → **Architecture Synthesis** (where computation happens) → **Java Backend Coding Technology** (how each component is implemented) → running software

Each volume derives its layer from the previous one's outputs, and each stands alone. The reading map: a practicing architect starts *here*; a methodologist or language-neutral reader starts with *Process-First Design*; a Java developer starts with *JBCT*. Two notes for readers arriving from the siblings. This book's answer sheet **supersedes** the Architecture Synthesis module in *Process-First Design* — the module remains a faithful design-time preview, and as of *PFD* 2.1.0 the two books ask the same nine questions; earlier *PFD* editions asked eleven, and *PFD*'s revision history carries the transition. And the series speaks in two registers on purpose: *PFD* and this book are continuous argumentative prose; *JBCT* is a working manual with exercises and checklists — a coding book and a methodology book serve different hours of the reader's day.

Shared vocabulary is defined once per concept across the series; where this book renames for self-description (the recovery triple's long names), the first use carries the crosswalk.

The derivations, worksheets, and audits in this book are engineering methods, not professional advice for any specific system; applying them to your systems is your engineering judgment exercised on your facts, and no warranty travels with the instruments.

Acknowledgments

Some debts in this book are structural.

Denys Poltorak — twice. His *Architectural Metapatterns* (CC BY 4.0) supplied the transition catalog that Chapter 10's edge list is revoiced from, with his blessing; and his adversarial full-read of this book's companion volume set the standard of review this book was written to survive. The stress-tester is the rarest kind of colleague, and the most valuable.

Yannick Loth — whose Independent Variation Principle and cohesion work arrived at change-driver partitioning independently, from a different starting point, with no shared lineage — the kind of convergence that tells an author the territory is real. His close reading of the companion volume sharpened both books.

Juval Löwy, whose volatility-based decomposition is the nearest ancestor of the change-driver stance; **Gregor Hohpe**, whose architect-in-the-first-derivative framing this book's subtitle argues with and alongside; **Martin Kleppmann**, whose physics this book's ledger presupposes on every page; and the **SEI's QAW and ATAM** lineages, whose two mechanized doors defined the middle room this book set out to furnish.

John Allspaw, the **VOID project**, **Štěpán Davidovič**, and **Google's CRE** writings — the reliability lineage Chapter 12 dissolves a slogan with, using the lineage's own best work.

The operators who publish — Stack Overflow's engineering blog, Shopify's, Discord's, and the UK's National Audit Office, whose institutional candor made a blind derivation of a government registrar possible at all. Evidence-based architecture writing exists because some organizations show their work.

And the early readers and critics of the articles this book grew from — the skeptics most of all. Several arguments in these pages were forged in public threads, and are better for every round.

[Ship note: names used with permission where quotation or attribution exceeds fair scholarly practice; permissions pass scheduled before release.]

Two Teams, Neither Wrong

Two teams build the same product: software that sells a ticket to a seat at an event. One team ships a single program talking to one database. The other ships a multi-region platform of subsystems trading events across a bus. They share almost nothing — not the topology, not the storage, not the way one part of the system talks to another.

Neither team made a mistake.

Everything in this book follows from that sentence, so it deserves a moment of pressure. If two architectures that share nothing can both be correct for the same domain, then the domain never determined the architecture. Something else did. In most teams, that something else is taste: the last conference talk, the case study about how a company a thousand times larger does it, the loudest voice in the design review. Architecture selection is the least standardized decision in backend engineering, and the industry has normalized this by giving it a dignified name. We call it trade-off analysis. Examined closely, much of it is preference with a vocabulary.

That "something else" has more candidates than taste. Team structure, the contracts that pay for the work, an appetite for risk — each genuinely bends what teams build, and an account that stops at "not the domain" owes them a place. This book's treatment is the same for all of them: written down, priced, and scoped, they become inputs like any other commitment; left unwritten, they draw boundaries nothing justifies — and Chapter 9 gives those boundaries a name — unforced positions — and a price.

There is a better account of the two teams. They had different **answers to the same questions**, and the architectures follow from the answers — mechanically. Both teams ran the same derivation; the inputs differed, so the outputs did. Architecture is not a choice you defend. It is an output you compute, and recompute when the inputs change.

That is the book's entire claim, stated as bluntly as it will ever be stated. The rest of the book earns it: this chapter runs the derivation once, in miniature, so you can watch it work before any part of it is formalized. The chapters after it take the machinery apart, run it on systems nobody asked us to derive, grade the results in public — misses included — and then turn to the harder problem: the system you inherited.

The debates are the tell

Consider what architecture discussion actually looks like where you work. Someone proposes services; someone counters with operational cost; someone cites an outage at a company neither of them works for; someone says "it depends" and the meeting ends. The same discussion reconvenes a quarter later with the same positions and no new information, because nothing in it was anchored to anything that could settle it. Architectural debate is the only technical activity we routinely conduct with no termination condition.

The debates are the symptom that exposes the disease. A debate without a termination condition is what you get when a decision has inputs nobody has written down and criteria nobody has agreed to. Strip the vocabulary away and most architecture arguments reduce to two people defending different defaults — and often, without noticing it, arguing about entirely different properties of the system. One is worried about deployment independence, the other about consistency across a boundary; both say "microservices." The argument cannot converge because it is not about one thing.

Watch what happens to the same discussion when an input sheet exists. "We should split the checkout path" stops being a position and becomes a claim with a checkable premise: which answer does the current structure fail to contain? If the answer exists — a stated target the current position cannot meet — the split becomes a calculation, and the discussion moves to whether the answer is real. If no such answer exists, the proposal dies without a meeting. Either way, something terminated the debate, and it was not seniority.

That termination condition is what this book builds: a procedure that makes most of the advocacy unnecessary.

Three kinds of book, two of which exist

Almost everything written about software architecture is one of two kinds. The first is the **catalog**: styles, patterns, reference architectures, trade-off matrices (a well-organized menu of options, with guidance that ultimately reduces to *pick one and defend your pick*). The second is **physics**: how replication actually behaves, what a queue actually absorbs, what consensus actually costs (the mechanics of the materials). The catalogs are genuinely useful; so is the physics, and the best of it is superb. Neither tells you what to do on Monday. A catalog gives you options without a selection procedure. Physics gives you mechanisms without a decision path. Between them sits the actual daily question — *which of these, for us, now* — and on that question both kinds of book go quiet, at exactly the moment the loudest voice in the review does not.

The gap is visible even in the most disciplined corner of the field. The Software Engineering Institute — the people who took architecture evaluation more seriously than anyone — built a workshop for eliciting quality requirements from stakeholders (the QAW) and a method for evaluating a candidate architecture against them (ATAM). Elicitation on one side, evaluation on the other. Between them, where the candidate architecture actually comes from, the guidance says: expert judgment. The most rigorous framework in the industry has a mechanized front door and a mechanized back door, and in the middle, a room where the experienced architect does something no one has written down.

That middle room is this book's subject. The missing kind of book is a **procedure**: elicit these inputs, run this derivation, get an architecture out, re-derive when the inputs move. With the middle room mechanized, the SEI's two doors become more valuable, not less — elicitation feeds the derivation, and evaluation becomes a check on its output rather than a substitute for producing it.

If a procedure sounds too ambitious for something as contested as architecture, it is worth remembering that hardware engineering had this argument, and settled it, in

the 1990s. Circuit design used to be schematic capture by expert hand: taste, experience, style, defended in review. Then logic synthesis arrived: a behavioral specification plus timing, area, and power constraints go in; a circuit comes out. Engineers did not stop mattering. Their judgment moved upstream, into the specification and the constraints — the inputs — and out of the gate-level netlist, where it had never belonged. Nobody today argues a netlist by taste. Software architecture is a specification-and-constraints problem of precisely the same shape, and it is still being argued gate by gate.

The analogy carries a disanalogy, named here so it stays ahead of its critics. RTL is a formal language and a timing constraint is a number; an answer sheet is elicited from people, who can game it. That is why the next chapter spends half its length on an entry gate that refuses unpriced answers — the gate is this method's informal type-checker, and the method is exactly as strong as the discipline at its input. History also counsels patience about adoption curves: synthesis produced worse-than-expert netlists for years before it won. The bet is on where the curve ends, and the hardware precedent is what the bet stands on.

Answers in, vector out

A procedure needs a defined input and a defined output. Both deserve a preview here, because the miniature derivation below uses them; both get a full chapter of their own.

The input is an **answer sheet**: the system's commitments, stated as numbers with scopes. What must this operation return within, and for which percentile of requests? How much of this data class may be lost, ever? Which reads must see the writer's own write, and which may lag? What does the peak look like, on which path? Which regulator can demand what, replayed from when? Each answer is a business commitment, not a technical preference — and each answer only counts once its cost is acknowledged. "We need 99.99%" is not an answer until someone has seen what a fourth nine costs per year and still wants it. An SLO you haven't priced is a wish.

The output is not a style with a name. It is a position on **six axes**, each with a small set of values, each value carrying real capabilities and real, always-on costs:

Axis	Values
Deployment topology	single deployable / multiple / unified runtime / serverless
Composition substrate	direct calls / event-based / streaming
Read/write model	unified / separated
State storage	current-state / event-sourced
Persistence	single shared / distributed / sharded / per-component / polyglot
Recovery	compensate-by-inverse / degrade-and-continue / design-out

"Microservices versus monolith" — the debate that consumes the industry — is one value on one axis. The other five axes exist whether or not you name them, and every

one of them is being set, by someone or by default, every time a system is built. Most preference wars are two people arguing about different axes without noticing.

One more property of the output matters before you watch it get computed: values apply **at a scope**. A system does not "use event sourcing"; a data class within it does, or doesn't. A system is not "CQRS"; one read path is separated, or none is. The named styles that catalogs trade in are bundles — several axes fixed at whole-system scope and given a marketing name. The vector unbundles them, which is what makes positions checkable one at a time.

The derivation, in miniature

Three rules run the whole thing. They get a chapter of formal treatment later; here they are as they operate:

1. **Start at the cheapest position on every axis** — single deployable, direct calls, unified reads, current-state, one shared store.
2. **Move an axis only when an answer is not contained by the current position.** An answer the cheap position already satisfies **presses** on nothing; it is **inert**. Both words are load-bearing for the rest of the book, always in exactly this sense: a demand *presses* an axis when it escapes what the current position contains, and an answer that presses nothing is inert — recorded, and rightly ignored.
3. **When you must move, take the cheapest value that contains the demand, at the narrowest scope that contains it.** Event-source one data class, not the system. Split one read path, not the world.

(There is a fourth rule, for the day two answers press one axis in opposite directions. The team below never needs it — which is itself the point. It gets its treatment where it belongs.)

Now the smaller of the two teams from the opening. One venue runs its own box office: one team, one region, one country. The answer sheet, in full:

- Buying a ticket: about 2 seconds at P95. The availability check: under 1 second.
- Bookings: tens per second on a busy night. Availability checks: somewhat higher.
- Service availability: 99.5% — hours of downtime a year, survivable for one venue, priced and accepted.
- Booking data: strictly consistent, zero loss on confirmed sales. Availability and price reads: eventual is fine.
- One currency, one legal regime. Daily deploys. A tight cost ceiling. Read-moderate load.

Walk the axes and watch what presses.

Deployment topology. What would demand a second deployable? Independent release cadence between parts of the system; independent scaling for a path whose load is shaped unlike the rest; a blast-radius boundary a failure must not cross. The sheet contains none of these: one team releasing daily as one unit, load in the tens per second everywhere, an availability target that a restart budget satisfies. Nothing presses. **Single deployable.**

Composition substrate. An event substrate earns its cost across deployment boundaries — temporal decoupling, burst absorption, fan-out — and this system has no boundary to cross. A queue here would add propagation lag and delivery semantics to paths that a function call serves in-process. Nothing presses. **Direct calls.**

Read/write model. Reads and writes share one shape; no read path carries its own contractual target; the read volume sits comfortably inside what the write model serves. A projection would be standing machinery — built, monitored, backfilled — against a problem no answer states. **Unified.**

State storage. One answer could move this axis, and it is worth naming because it so often gets bought by accident: a demand to *replay* history — to reconstruct what the system believed at a past moment, under the rules of that moment. No such answer exists here. Confirmed sales must survive; nothing must be replayed. **Current-state** — and should an auditor ever ask *who changed what, when*, an audit log written in the same transaction answers it without turning every read in the system into a projection.

Persistence. Two seconds at P95, tens of writes per second, strict consistency on bookings. A single store contains all three — and hands over the strict consistency for free, through ordinary transactions, the way single stores have done for fifty years. **Single shared store.**

Recovery. The one axis the domain itself forces, because buying a ticket moves money through sequenced steps: reserve the seat, authorize payment, confirm. Steps will fail mid-sequence; the axis asks what the domain offers for unwinding them — a question answered by the domain's shape rather than by the sheet's numbers, the second input class Chapter 2 makes formal. Here, every step has a defined inverse: release the seat, void the authorization. **Compensate by inverse.**

The vector, assembled: *single deployable / direct calls / unified reads / current-state / single shared store / compensation.* A boring program and one database.

Read the walk again and notice what did all the work. **The team didn't choose simplicity. The derivation found zero uncontained demands.** Every answer sat inside the cheapest position, so every axis stayed put. That is not minimalism as a virtue, or restraint, or good taste. It is arithmetic — and the same arithmetic cuts the other way. Copy the multi-region platform's architecture onto this answer sheet and every axis moved without a forcing answer becomes a standing bill: machinery to operate, failure modes to debug, staleness to explain — paid monthly, for capabilities no answer demands. That is how small platforms drown, and the drowning is usually described afterward as "we over-engineered," as if it were a mood. It was a computable error.

And the other team, the multi-region platform from the opening? Same procedure, different sheet: a contractual 200 milliseconds at P99 on the quote path across regions, on-sale bursts in the hundreds of thousands per minute against a fixed number of seats, a regulator requiring replayable price history, sovereignty rules pinning data by law. Each of those answers breaks containment somewhere specific, and only that axis moves, at only that scope. The full walk — and the systems between the two extremes — is Part II's work. What matters here is the shape of the explanation: **both architectures are outputs of one procedure, fed different answers.**

When the answers change

Systems do not hold still, and this is where the procedure stops being a design-day tool and becomes something else.

The venue grows. It joins a regional network; a partner contract now specifies availability checks under 500 milliseconds; the read load goes heavy. What happens to the architecture? Precisely one thing: the read path — and only the read path — now carries an answer the current position no longer contains. The derivation moves one axis, at one scope: read replicas behind the same unified model, because the staleness budget allows them and they cost less than projections. Everything else stays. No migration program, no target-state architecture, no two-year roadmap. **The next correct step, computed from the new answers.**

That phrase is the book's subtitle, and this is the reframe it names: an architecture is not a state a system is in. It is a **derivative**: the next correct step from where the system stands, given the forces acting on it now. A greenfield architecture is the derivative taken from a blank page. The system you inherited — running for a decade, carrying decisions nobody alive remembers making — is the same computation from a harder starting point. Part III takes that reframe seriously, all the way to a real inherited system with a public audit trail.

What the book owes you now

A claim like "derived, not chosen" creates obligations, and the book accepts all of them.

First: **where do the questions come from?** A derivation is only as honest as its inputs, and a question list handed down without justification is just taste moved upstream. The next chapter holds the questions to a membership criterion — every one has to demonstrate that its answer can force an axis on its own — and the sheet's count is that test's output, free to grow the day a new demand earns a seat.

Second: **why these axes, and what do the values actually buy?** Every axis value carries capabilities and always-on costs, and "cheapest containing" means nothing until the costs are on the table. That ledger is Chapter 3, and the derivation rules that consume it are Chapter 4, with the verification that closes the loop in Chapter 5.

Third, and this is the obligation most methodology books quietly decline: **proof against systems we do not control.** Part II re-runs ticketing at three scales with every step cited. Then it does something stricter: derives four real systems — Stack Overflow, Shopify, Discord, and a British government registrar — from their public commitments alone, with predictions registered before looking at the outcomes, and grades the results in the open. The procedure gets some things impressively right. It also misses, and the misses are kept, because a method that never fails is a parlor trick, and what a miss teaches is worth more than what a hit confirms.

Fourth: **naming where judgment remains.** A method that claims to mechanize everything is selling something; this one maintains an explicit inventory of the decisions it hands back — targets, recovery ties, contradiction choices, product picks — and Chapter 12 is that inventory, kept as carefully as the mechanics.

Two disciplines run underneath all four obligations, stated once here and honored throughout. Claims wear their grade: **derived** (follows from cited answers by named rules), **empirical** (graded against a documented outcome), **heuristic** (works across the validation record, mechanism plausible), or **contextual** (true in a scope, stated with it), and the provenance labels you will meet in the evidence chapters ([reconstruction], UNVERIFIED, registered-and-graded) are this legend at work. And the method's assumptions are on the table now, not discovered later: enterprise backend systems in the percentile regime; answers that can be elicited and priced from someone accountable for them; ledger entries that state mechanism physics, not fashion. Where an assumption fails — hard real-time correctness, unpriceable answers, novel mechanisms without operational record — the method says so instead of stretching.

The two teams from the opening never argued, because they never met. Your teams meet every quarter. The next chapter starts where every derivation starts — with the questions, and with the discipline that makes answers worth deriving from.

The Answer Sheet

The meeting is scheduled for an hour and runs ten minutes over. The product owner wants the new platform "highly available — let's say 99.99%," "real-time where it matters," and "scalable, obviously." The architect writes all three on the whiteboard. Everyone nods; the meeting ends; the requirements document says exactly what the whiteboard said. Six months later the platform carries a cross-region consensus store nobody can operate, a queue on paths that needed function calls, and a latency target no one can state from memory — and every one of those purchases traces back to that whiteboard, where three wishes were recorded as answers.

Chapter 1 claimed the architecture follows from the answers. That claim puts enormous weight on the answers, and this chapter is where the weight lands. A derivation is only as honest as its inputs: garbage answers derive garbage vectors with perfect mechanical integrity. So the input side needs two things a requirements meeting never produces on its own — the right questions, and a gate that keeps wishes from passing as answers.

It also needs something subtler, which this chapter does in the open: a justification for the questions themselves. A question list handed down without argument is taste moved upstream — the loudest voice replaced by the author's voice, which is no improvement. Every question below has to demonstrate its seat: its answer must be able to force an axis on its own. The list was not born this clean; it is the survivor of an audit against this method's own derivation record, and its count is an output of that audit rather than a choice.

What a question must do to earn a seat

The derivation gives questions a job description. An answer participates by *pressing* an axis toward a value or *pruning* values an axis can no longer hold. So the membership criterion writes itself: **a question earns its place only if its answer can press or prune at least one axis independently of every other question's answer.** A question whose answers never move anything is decoration. A question whose pressure always arrives bundled with another question's is a duplicate wearing different words.

Nine questions hold seats under that criterion, each audited against every derivation this book runs. The count deserves one sentence of standing law rather than a history: it is an output, and if a tenth demand ever demonstrates independent pressure — pressing or pruning an axis no current answer reaches — the sheet grows. The criterion outranks the number.

The questions organize by what their answers *do* in the derivation, because that is the property that matters when you hold the sheet. Driver modes cover them.

Prune mode: the correctness answers

Prune-mode answers are binary. They do not trade against money or cleverness; a value that cannot honor them is struck, and no amount of hardware buys it back.

The consistency contract — per data class and per path: which reads must be strict, which may lag behind writes and by how much, who must see their own writes. This is the question people answer with a shrug and pay for with an incident. The contract is per data class because one system legitimately carries several: the booking that must be strict and the availability display that tolerates thirty seconds of lag are different contracts in one product.

The loss budget — per data class: how much of this data may be lost, ever, and how long must it be kept. A recovery-point objective of zero on confirmed sales is a prune-mode fact: values that cannot durably commit before acknowledging are simply gone from the board.

External constraints — the regulator's and the contract's reach: what must be auditable, what must be *replayable*, what must be *erasable*, where data must reside, which mandates strike values outright. One decomposition inside this question does more damage prevention than any other in the sheet, and it gets its own section below.

Multi-X — countries, currencies, tenants, regions, versions. Two different outputs hide here: partition *gifts* (tenants that never interact are a natural shard key — Part II shows one carrying a hundred-billion-dollar platform) and legal *pins* (data that must stay in-country by statute). One question, because both arrive in the same breath of business fact; two readings, because one gives capacity away and the other takes options off the table.

Select, isolate, split, bound

The time budget (select mode) — per operation, and *shaped*, never a scalar: percentiles and tails for request paths, soft maxima where a human waits, completion windows where a batch must land inside a business deadline. Latency selects among surviving values rather than striking them, and it presses only near a physics floor: a 2-second budget over a 2-millisecond path forces nothing, a 50-millisecond contract over a cross-region hop forces everything. The shape catalog doubles as a scope detector: an answer with a *hard* maximum as a correctness criterion — a deadline whose breach is a system failure, at hard-real-time strictness — is the tripwire that says this book's methods stop applying. Enterprise backends live in percentiles; anti-lock brakes do not, and the sheet must say so at elicitation time rather than let the derivation discover it.

The failure budget (isolate mode) — per operation: the error budget and the criticality that sets it. A 99.5% target is forty-four hours a year, and a restart strategy fits inside it; a 99.99% target is fifty-three minutes, and whole classes of design stop fitting. Availability starts biting past roughly three nines on a named path, and what it bites is blast radius: the axis moves it forces are isolations.

Load (split mode) — magnitude at steady state and peak, shape per path, concentration, and window. This is the merged question, and the merge taught a discipline: load

answers are only meaningful *per path*, because "the system is read-heavy" is routinely true at one tier and false at another: Part II contains a system that is read-dominated at the HTTP tier and write-majority at the store, simultaneously, and any sheet that recorded one number for it would be wrong twice. Load presses only on *divergence*: uniform load, however large, presses sizing rather than structure. The shape vocabulary that decides what a load answer can force — volume (sustained magnitude), burst (a peak with a tolerable settling delay), contention (many actors, one record), deadline (finish inside a window) — belongs to the ledger, where each shape meets the mechanism that contains it.

Release structure (bound mode) — cadence divergence between parts of the system, and deploy safety; one of this question's usual bundles is a fake driver so common it needs its own paragraph below.

The cost and capacity envelope (bound mode) — money, and the question that sounds too small to matter and decides more architectures than latency does: *who operates this?* Four engineers and no operations team is an answer that moves axes. Bound-mode answers rarely press a single move; they set the envelope every other resolution optimizes inside.

The entry gate: answers are purchased, not collected

The whiteboard meeting failed before any question was asked, because it treated answers as things people say. The derivation treats answers as things people *buy*. Five disciplines make up the gate.

Priced. Chapter 1 planted the flag on unpriced SLOs; the gate is where pricing happens. A fourth nine costs a number someone can compute — in redundancy, in on-call, in correlated-failure engineering — and the pricing drill is one question long: *what does the business do differently in the fifty-third minute of annual downtime?* If the honest answer is "nothing," the real target just revealed itself, one nine lower and an order of magnitude cheaper. Most nines dissolve under this question. The ones that survive are real, and the derivation treats them with full respect.

Scoped. Scope lives inside the question — per operation, per data class, per path — never around the system. One system-level number is how bare adjectives sneak back wearing digits. The banned vocabulary is worth naming: "scalability," "high availability," "performance" (words that hide both the scope and the shape of a demand). The sheet forces the honest versions: *which path*, *under what load*, *within what budget*.

Decomposed. Answers arrive as bundles, and bundles press falsely. The canonical fake driver: "the teams need independence." Decomposed, it splits into ownership boundaries (which module boundaries contain at zero deployment cost) and release-cadence independence, which is a genuine topology driver that most teams, asked directly, do not actually want: they want to stop stepping on each other, and one deploy train with owned modules delivers that. The bundled version buys a service fleet; the decomposed version buys a directory structure. The same discipline dismantles "we need audit," which is where the most expensive accident in the storage axis lives, and it splits three ways, because a third demand hides behind the same meeting language. **Audit** asks *who changed what, when*: answerable by an audit log written alongside

current state. **Replay** asks *what did the system believe on March 3rd under March 3rd's rules*: answerable only by event-sourced history. And **erasure** — the right to be forgotten, a statutory demand wherever personal data lives — runs *against* both: it requires that specific facts become irrecoverable, which strikes exactly the append-forever machinery replay buys. Three demands that sound alike in a requirements meeting force three different storage answers, and one pair of them can genuinely contradict: replay-required and erasure-required on the same data class is a statutory collision the derivation must surface, not paper over. The regulated industries' folklore that "compliance requires event sourcing" is this three-way decomposition skipped, at seven figures.

Triaged. Two things masquerade as answers and belong outside the sheet. Business-process deadlines bind the *requester's* clock: a statutory 14-day filing window is a real obligation that no latency mechanism can address and no architecture axis feels: it re-enters later as a calendar constraint on transformations. Only system-clock targets press. And observed failures — last quarter's outage, the incident that started the whole initiative — are evidence about failure domains, never demands: they earn a place in the derivation only if the business converts them into a number it will pay for, which is the gate doing its job on trauma the way it does on wishes.

Surfaced early. When two stakeholders answer the same question differently at the same scope — the CFO wants every report strictly current, the COO won't fund the store that could deliver it — the sheet has caught a business contradiction wearing an engineering costume. Catching it here, where renegotiation costs a meeting, is the cheap version of a discovery that otherwise arrives in production. What the derivation does when a contradiction survives all the way through is Chapter 8's subject, and it is the method at its most useful.

The second row source

One kind of input deliberately lives outside the nine questions, because no stakeholder answers it — the domain itself does, through the analysis work this book's companion methodology owns. Two families of fact arrive this way.

Domain-shape facts, for every operation that changes the world: does a *defined inverse* exist (a reservation releases, an authorization voids — but an email does not unsend)? does the value *decay* (a seat hold that expires is a fact with a half-life)? can the operation be *reshaped* so a whole class of failure cannot occur (made idempotent, commutative, append-only)? These drive exactly one axis — recovery.

Change-driver facts, for the system's rules and data classes: how often, and under whose control, does the logic governing this data class change? A benefits platform whose rules move with political weather, a pricing engine whose rules move with the market, a tax calculation whose rules move annually by statute — these are facts about the domain's *volatility structure*, and they come from the same change-driver analysis that PFD builds its entire design method on. The analysis method is PFD's; the sheet does not require the book — the question as stated, asked of every rule set and data class and answered by whoever owns the roadmap, produces the row this book needs. They rarely press an axis alone; their power is in combination — volatility meeting

a continuity answer is what forces isolation between a paying path and its changing rules, as Part III's largest case shows at national scale. This is the precise seam where PFD's central concept feeds this book's sheet: the design method finds the change drivers; the architecture method prices what they press on.

Both families are analysis output, not elicitation answers, and pretending they are a tenth and eleventh question would misstate where they come from. The sheet has nine questions and a second row source with two kinds of row.

What a filled sheet looks like

The venue's sheet in Chapter 1 is the miniature. A real one runs a few pages: every answer a number with a scope and a citation to whoever committed to it, UNKNOWNs recorded as UNKNOWNs — an honest UNKNOWN derives a null position and a note, which beats a confident guess deriving machinery — and the domain-shape facts listed per effectful operation. It reads less like a requirements document and more like a term sheet, which is what it is: the business's commitments, in the only form a derivation can consume.

The sheet also defines what it means to disagree with this book productively. Every dispute about an architecture derived from a sheet is a dispute about a row — a price someone didn't accept, a scope drawn wrong, a decomposition missed. Arguments about rows terminate. That was the promise of Chapter 1, delivered by structure.

Answers press; something has to receive the pressure. Each axis holds a small set of values, each value provides specific capabilities through a specific mechanism at a specific ongoing cost — and "cheapest value that contains the demand" stays a slogan until those costs are on the table. The next chapter builds the table.

Instruments introduced: the nine-question sheet · driver modes (prune / select / isolate / split / bound) · the membership criterion · the entry gate (priced, scoped, decomposed, triaged, surfaced) · the audit / replay / erasure decomposition · the second row source (domain-shape facts, change-driver facts).

The Six Axes and the Ledger

A chat platform's busiest channel goes hot: a hundred thousand people watching one conversation, every one of them pulling the same recent messages from the same database partition. Concurrency climbs, latency cascades, the partition melts. The team's first instinct is the industry's first instinct — shard harder — and it cannot work, structurally: those reads want *one channel's* rows, and one channel has one home no matter how many shards exist. What contains the melt is a thin layer in front of the store that notices a hundred thousand identical in-flight reads and sends the query *once*. The fix costs a fraction of the resharding project it replaced, and no catalog of architecture styles contains it, because it lives below the altitude catalogs describe: it is a fact about what mechanisms can contain.

This chapter is a book of such facts. The method's geometry has three spaces: the **demand space** Chapter 2 produced, where commitments live as numbers with scopes; the **position space** Chapter 4 will walk, where every architecture is a vector on the six axes; and between them the **selection space** this chapter maps — what each position on each axis *provides*, through which *mechanism*, at what *always-on cost*. Classic architecture literature works in this middle space with quality-attribute vocabulary and drowns, because "-ilities" are not decidable. The catalogs skip the space entirely, jumping from demands to named styles by narrated judgment. The derivation becomes checkable at exactly the moment this middle space becomes explicit: **selection is the cheapest vector whose capability envelope contains the demands** — a sentence that means nothing until capabilities and costs are written down, and everything after.

The written-down version is the **ledger**. Its discipline, before its contents:

- Entries are **mechanism physics** — what a projection costs, what a queue absorbs, what quorum commit cannot avoid (the round trips a write pays to reach agreement across a majority of replicas). They are never compatibility rules between styles; interactions surface inside derivations, as contrast.
- Values are **atomic**. "Hybrid" and "mixed" are compositions produced by scope splits, priced as parts plus the boundary between them.
- Values apply **at demand scope**. Event-source one data class; separate one read path; extract one component. A value applied wider than its demanding scope is unforced cost — a definition of technical debt this book will use repeatedly.
- Costs are **always-on**. A mechanism bills for existing — operation, evolution, failure modes — whether or not the capability is exercised this quarter.
- **Containment is a claim about a bound.** A value or mechanism *contains* a demand when it holds the demand's stated number — at its percentile, over its window, under its declared shape — at the demand's scope, at the mechanism's always-on cost. The claim is exactly as probabilistic as the bound it serves: a replica pool contains a read-volume demand at P95 and at a named staleness, not absolutely; an admission gate contains a contention demand up to its configured arrival rate and not beyond. Whatever assumptions the bound carries, the

containment claim inherits — which is why every containment survives or dies at the exit gate's arithmetic rather than on the ledger's word.

Demand shapes: what kind of pressure is this?

Before any axis question, a load answer must declare its shape, because shape selects the containing mechanism family — and the industry's standing error is buying volume mechanisms for contention problems. One diagnostic separates them: **would a second copy help?**

Volume says yes. Sustained magnitude spreads across copies, caches, shards; volume is the shape capacity answers.

Contention says no. Many actors, one record — one seat at the on-sale moment, one hot channel, one flash-sale inventory row. The contended record has one home regardless of fleet size, so containment means governing arrivals: admission control and fair queueing on the write side, request coalescing on the read side, or reshaping the operation so the conflict cannot occur. Sharding a contention problem buys hardware and keeps the melt.

Burst is a peak with a tolerable settling delay, and its mechanism is a buffer: the queue absorbs what the deepest dependency never sees. The definition contains its own test — a peak that must be served *synchronously* is volume at the peak number, and calling it a burst just moves the failure two weeks later.

Deadline is "finish inside the window," a batch shape: payroll by the 25th, filings by year-end. Deadlines are answered by windowed throughput and resumable work, and latency mechanisms are irrelevant to them — with the Chapter 2 triage in force, because most deadlines in a sheet bind the requester's clock and press nothing at all.

The shape taxonomy is deliberately open at the front. New workload families arrive wearing new clothes and old shapes: a GPU serving one large model is *contention* (one resource, many claimants — admission and batching, never naive replication of what cannot be copied cheaply); token streaming is the *streaming* substrate's shape at session scope; a training run against a cluster deadline is *deadline*-shaped with checkpointing as its resumable-batch mechanism. The diagnostic questions do not change; the examples will.

Containment rungs: the moves that are free of architecture

Some mechanisms contain demands without moving any axis, and pricing them first is what keeps derivations minimal. The rule that governs them: a tier that owns **no business logic and no data of record** — a load balancer, a cache, a coalescer, an admission gate — is a containment mechanism, invisible to the vector. Systems accrete many such tiers; their architecture does not change thereby, and a vector that counted them would overstate every real system it described.

Rung zero is hardware. Sizing the cheapest position — more memory, more cores, the whole working set in RAM — contains volume and latency pressure with zero new

mechanisms, which by this chapter's selection logic makes it the first bid on every containment, and one of the great engineering cultures of the web ran on that rule as an explicit company value. The rung has a ceiling, and the ceiling is economic rather than technical: it stops when the working set or the write rate outgrows what one box's price curve buys, and past that point every further dollar purchases less than the mechanism it was avoiding. The price curve itself is substrate-dependent, and the rung must be priced on *yours*: on owned metal the curve is gentle and the rung famously long; on cloud pricing it steps at instance-size cliffs, bills egress separately, and caps out earlier: the same rule, run against a different ledger row, sometimes returning a different answer. What does not change is the rule's position: first bid, priced before any axis moves.

Read pressure climbs a fixed chain: cache → coalescing → replicas → projections. Each rung contains a different shape: repeated reads of hot values; concurrent *identical in-flight* reads; same-shape read volume past one node; and only at the top, reads whose *shape itself* diverges from the write model. The top rung is the axis move. Everything below it is not, and the chain's discipline is that you climb it rung by rung, stopping where your citations stop. Part II grades this chain against three industrial systems that stopped at three different rungs — each stop forced by that system's answers — and against the fashion that begins at the top rung by default and calls it best practice.

The axes

Six axes, each with atomic values, each value a **provides / mechanism / costs** entry — and the entry's three fields are exactly the joints the procedure will articulate: prune strikes a value when *provides* cannot meet a correctness demand; press tests demands against what *provides* holds at scope; resolve counts *mechanisms*; and *costs* is the always-on bill the whole selection minimizes. A richer schema — requires columns, excludes columns — would add nothing the three fields do not already encode: exclusion is the absence of a provides, and a prerequisite is a mechanism, priced like any other. What follows renders the ledger at chapter altitude — the full entries live in the reference cards.

Deployment topology — *single deployable / multiple deployables / unified runtime / serverless.* This axis prices two countables, kept apart because its costs split between them: the **artifact**, versioned and released as one — release coupling and the delivery plumbing bill here — and the **runtime unit**, an instance group shaped and scaled as one — scaling shape and runtime blast radius bill here; the network inside a crossing bills to neither, only to the boundaries the composition substrate actually crosses, and is largely pre-paid where the substrate is already event-based. The single deployable — one artifact, identical instances — provides in-process calls on every internal path, one operational surface, and one composition in production — the composition you test is the composition that runs — at the cost of whole-system blast radius and whole-unit scaling; its modular form adds ownership boundaries at zero deployment cost, which is the fact that dissolves most "we need services" pressure once Chapter 2's decomposition has run. Its **role-selective form** — the same artifact, handlers or process classes enabled per instance group, a form with decades of record as web-and-worker

processes from one codebase and database node roles from one binary — buys back per-role scaling envelopes and inter-pool blast isolation with no second artifact, at the price of the activation machinery becoming a mechanism of its own: role configuration, per-role capacity, knowing which instance runs what. What no single artifact buys is release independence: one artifact is one train, disabling a handler at runtime is a kill-switch — recovery machinery, not a cadence — and waking a dormant handler on a running instance takes the isolation back, one process envelope now shared. The train itself is a bill that grows with the codebase — one build, one test gate every team's merge waits behind. Module-aware builds and affected-only test selection contain it a long way, a containment rung of its own; what remains when they cap out is a cadence demand, pressing the axis through the answer sheet rather than around it. Multiple deployables provide what only artifact multiplicity can: independent release cadence, independent toolchains and dependency evolution, blast isolation that holds at deploy time too. Their cost is a network inside every crossing — a latency floor, partial failure as an internal concern, consistency across the boundary decaying from transaction to protocol — paid in full where calls are direct, and mostly pre-paid where the substrate move already put a broker between the parts. Their second cost is the delivery plumbing, multiplied: each artifact is its own pipeline, its own release process, its own dependency stream to keep patched — and its own row in a version matrix, because independent cadences mean production runs a mixture of versions through every rollout window. The composition that was tested is no longer the only composition that runs; cross-artifact compatibility stops being an event and becomes a standing bill — contract tests, deprecation windows, the N-by-M pairs. The unified runtime holds packaging open — one-or-many decided at deploy time without rewiring — at the cost of a platform dependency; the role-selective form is its hand-rolled ancestor, and what the product adds is the same deploy-time freedom for *direct-call* composition. The platform class is young; the pattern is not. This book's author builds one such runtime, disclosed here precisely so the derivations can be checked for favor: none requires it. Serverless provides per-invocation scaling to zero at the cost of cold-start tails and per-invocation pricing that crosses over against sustained load.

Composition substrate — *direct calls / event-based / streaming.* Direct provides the lowest latency and read-your-effects immediacy at the cost of temporal coupling: the callee must be up *now*, availability multiplies down the chain, bursts arrive unbuffered at the deepest dependency. Event-based buys temporal decoupling, burst absorption, and fan-out, and its price deserves stating in full because it is so often paid unknowingly: propagation lag on every consumer view, delivery semantics that force idempotent consumers, ordering only per key — and the between-steps state becomes durable, named, and *operated*. Streaming provides ordered, replayable, consumer-paced consumption for the one data class whose volume earns a partitioned log, at the cost of partition-key design becoming load-bearing.

Read/write model — *unified / separated.* Unified provides read-your-writes for free and zero projection machinery; the entire containment chain above lives inside this value, which is why it survives far more load than its reputation suggests. Separated provides independent scaling and shape for one read path, through projections, at the cost of a staleness window, machinery to build and backfill, and dual schema evolution

— a price worth paying exactly when a read path carries its own contractual target *and* its own shape, and worth refusing otherwise.

State storage — *current-state / event-sourced.* Current-state provides the read as the state; paired with an audit log written in the same transaction it answers every *who-what-when* question, which is why Chapter 2's three-way decomposition guards this axis. Event-sourced provides genuine replay — state at any past moment, *why* under that moment's rules — at a cost that compounds forever: every read a projection, event schemas versioned for life, unbounded growth, and a model the team must learn to think in. The value is real and the demand that earns it is rare; the ledger's job is keeping the two matched. And one statutory demand runs *against* this axis's expensive value: erasure. Where the law requires that personal data become irrecoverable, append-forever machinery meets its counter-demand, and the containment mechanisms are their own small ledger row: scope exclusion first (keep personal data out of the immutable stream, referenced from a mutable store), crypto-shredding (encrypt per subject, erase by destroying the key), tombstoning where the store supports honest deletion. Replay-required and erasure-required *on the same data class* is a genuine contradiction with a statutory source: the derivation surfaces it and hands back the menu, exactly as Chapter 8 shows.

Persistence configuration — *single shared / distributed shared / sharded / per-component / polyglot.* The single shared store provides cross-component transactions for free: strict consistency, delivered by fifty years of engineering, at the cost of a shared ceiling and shared blast radius. The distributed shared store holds a unique position: it is the *only* value in this ledger that provides strict transactions across regions with zero data loss on regional failure, and its price is physics, a write floor of cross-region round trips times quorum, which no vendor tunes away. Sharded provides write scaling past one node when the demand set carries a natural partition key; its cost is that cross-shard transactions effectively cease and the key becomes load-bearing. One escalation deserves its own entry: when volume sharding compounds with blast-radius isolation — one tenant's disaster must not touch another — the shard boundary widens to the **full stack**, a complete isolated instance of the system per shard: cells. The cost is the whole stack duplicated per cell and cross-cell features structurally forbidden, and the prohibition is the value working, as Part II's hundred-plus-cell example shows. Per-component and polyglot provide independent evolution and stores shaped to their data, at the cost of transactions across components decaying to protocol and one operational competency per store technology.

Recovery — *compensate-by-inverse / degrade-and-continue / design-out* — the axis with no null, because every effectful operation must answer it, and the axis whose inputs are Chapter 2's domain-shape facts rather than anyone's preferences. It is also the axis a careful reader flags as the odd one out: the other five position *structure*; recovery prescribes *behavior under failure*. The asymmetry is real, and the seat is earned under the same criterion as the rest — systems with different recovery answers differ structurally before any technology is chosen: compensation paths are real code with real tests, design-out reshapes the domain model itself, degradation demands bounded windows with visible state. The exit gate's failure column consumes exactly this axis's choices, per operation. An axis is a dimension along which systems

must differ; nothing requires the dimensions to be the same kind of thing. Readers of this book's companions know these three as **the recovery triple**: BER, FER, and design-out; the classes are identical, and the short names remain the series' compact notation (backward recovery compensates, forward recovery degrades and continues) while the long names carry the prose, because a name that says what you do beats one you have to look up. Compensation provides restored consistency where the domain offers an inverse, at the cost of compensation paths that are real code with real tests, and where the inverse must be *built*, it is a use case of its own with its own targets. Degrade-and-continue provides forward progress where staleness or decay is tolerable (defaults, queues-for-later, values that expire) at the cost of degraded windows that must be bounded and visible. Design-out provides the strongest guarantee available anywhere in this book: the failure class stops existing — idempotent writes, commutative operations, append-only records, a structural constraint that makes the double-booking unrepresentable — at a price paid once, in the shape of the model. Where design-out is available it embarrasses the alternatives, and checking for it first is a standing rule.

The boundary has a price

Compositions appear throughout the values above — separated *at one path*, sharded *at one scope* — and every composition contains a boundary, which the ledger prices like any mechanism. A scope split pays: a contract, versioned and negotiated evermore; consistency across the seam decaying from transaction to protocol; a translation layer answering to both sides' changes; an operational seam where deploys, monitoring, and incidents must be correlated, and for persistence splits, one durability story per store. This entry cuts both ways. It is why splits must be forced rather than fashionable, and it is why, when Chapter 4's conflict rule finds pressures at genuinely different scopes, splitting *at that boundary* is honest engineering: the price buys separation the demands actually require.

The selection rule

The ledger closes with the rule that consumes it, and the rule is short:

1. Start at the **null vector** — the cheapest value on every axis: single deployable, direct calls, unified reads, current-state, single shared store.
2. Move only when a demand is **uncontained** — and check the rungs before conceding that it is.
3. For prune-mode demands, test **scope exclusion** first: narrowing where the demand applies can beat hardening any axis — Part II contains a payment-card mandate contained by routing card data around the entire system, which thereby never entered the mandate's scope at all.
4. Among containing values, prefer the one introducing the **fewest new mechanisms** — each is an ongoing bill, and mechanism-counting is how "cheapest" stays honest when prices resist arithmetic. This is the precise sense of Chapter 1's "arithmetic": not that costs add like numbers — they do not — but that the comparison is mechanical, counting mechanisms rather than weighing taste.
5. Apply the value at the **narrowest scope that contains the demand**.

Why six axes and not sixteen? The same criterion that governed the questions, mirrored: an axis earns its seat only if systems with different answers must differ *structurally* along it before any technology is chosen. Candidates that fail collapse into technology choices — products, languages, clouds — which matter enormously and are not architecture. The strongest rejected candidate deserves its rejection shown, because it is the one every reviewer proposes first: **security topology** (trust boundaries, tenancy isolation, authentication placement). Run the criterion: the demands it carries route through axes that already exist: tenancy isolation is blast-radius isolation and arrives at cells; data-protection mandates are prune-mode external constraints and arrive at scope exclusion or erasure mechanisms; what remains after those routings (which gateway, which identity product, mTLS or tokens) is mechanism and technology choice, real work that no *structural* position decides. Two systems with different security answers do not differ along a seventh axis; they differ along the six, plus their technology sheet. The other candidate every reviewer proposes is **team topology** — the org chart, Conway's Law offered as a seventh axis. Run the criterion: team ownership is a real pressure, and the deployment entry above already contains it — module boundaries at zero deployment cost, with a genuine split forced only when release cadences diverge, a demand already on the sheet, pressing an axis that already exists. A team structure that splits a system no answer divides is not a missing dimension; it is an unforced position, Chapter 9's subject. Two systems with different org charts and identical answers derive the same vector; the org chart that overrode the derivation left debt behind, not a new kind of structure. The count is an output here exactly as it was in Chapter 2: the criterion is the theory, six is its current result, and the result is empirical — earned against the derivation record, not proven from first principles. A completeness proof for a design space is not on offer, here or anywhere. If a future derivation finds a security or organizational demand that presses nothing existing and forces structure of its own kind, the criterion outranks the count here too. That is what it is for. The six above have survived every derivation this book runs, one of them by surviving an audit this book nearly retired it with; Part II tells that story against a live system, because an instrument's near-death is evidence about the instrument.

Demands from Chapter 2, capabilities and costs from this one. What remains is the procedure that walks them against each other — including the moment two demands press one axis in opposite directions, which is where most methodologies wave their hands and this one runs a test.

Instruments introduced: the three-spaces model · the ledger and its discipline · demand shapes and the second-copy diagnostic · containment rungs and the read chain · the six axes' values · the boundary cost · the selection rule · the axis-membership criterion.

The Derivation

An order platform's design review has been stuck for a month. The checkout lead insists on strict consistency and a transactional store — money moves, carts commit, nothing may be half-true. The analytics lead insists on eventual consistency and cheap horizontal reads — dashboards scan millions of rows and nobody cares about the last ninety seconds. Both cite the consistency axis. Both are right. The review treats this as a conflict to be won, schedules another meeting, and escalates to the CTO, who will decide it by seniority, which is to say by taste.

The month was wasted on a question that has a mechanical answer, because the two demands never actually met: one attaches to the checkout path, the other to the reporting path. They press the same axis *at different scopes*, and pressures at different scopes are an instruction, not a conflict — the system splits at the boundary between them, and each side gets the value its own demands force. The meeting was arguing about which half of the truth would govern the whole system. The derivation's answer is that nothing has to govern the whole system; that is what scopes are for.

This chapter assembles the full procedure — the two instruments from Chapters 2 and 3, plus the resolution logic that handles everything the miniature in Chapter 1 was too small to need. The procedure has a name, `next_step`, chosen for what it computes: given where a system stands and what the answers currently are, the next correct position. Greenfield is the special case where the system stands nowhere.

The procedure

Step 0 — the null vector. Every derivation starts from the null vector — the cheapest value on every axis, exactly as Chapter 3's selection rule defined it. Recovery has no null; it is decided per effectful operation in step 4. For greenfield, the starting position *is* the null vector; for a living system, the starting position is wherever the system stands, and everything else proceeds identically — a symmetry Part III spends three chapters on.

The null vector earns a defense, because starting cheap sounds like an aesthetic. It is a measurement precondition. Pressure is only detectable as *escape from containment*, so containment must start somewhere defined, and it must start at the bottom: start anywhere higher and the unused capability is invisible — nothing presses against a wall that was already built, and you cannot distinguish the wall that earns its cost from the wall that is simply there. Every axis the null vector holds is a claim the answers are free to refute. That is the correct relationship between a method and its inputs.

Step 1 — normalize. The entry gate from Chapter 2, run to completion: every answer priced, scoped, shaped, decomposed to mechanism level, triaged. Nothing new here except the insistence that it happens *before* any axis is discussed — a design meeting

that starts at “should we use services” has skipped the step where most of its agenda dissolves.

Step 2 — prune. Correctness answers strike values. The consistency contract, the loss budget, residency pins, and mandates that survive scope exclusion each remove from the board every value whose ledger entry cannot provide them. Pruning is binary and cheap, and it goes first because negotiability orders the work: correctness cannot be bought back, physics can only be respected, economics optimizes inside what remains.

Step 3 — press. For each remaining demand, check containment against the current vector’s capability envelope. Contained demands are **inert** — recorded nowhere, pressing nothing. This is the step where the regime insight operates: a driver presses only when it crosses what the current position can hold, which is why most answers on most sheets press nothing, and why that outcome is a result rather than an anticlimax. An uncontained demand records a pressure: which axis, toward which value, at which scope, through which mechanism.

One check inside this step earns emphasis because a live derivation in Part II turned on it: pressures must be tested **in combination**, not only row by row. Two answers that are individually contained can jointly clear a bar neither reaches alone — a read volume that replicas would contain, meeting a read *shape* that replicas cannot serve, forces the projection that neither answer forced by itself. The pressure pass reads the sheet as a system, not as a list.

Step 4 — resolve. Pressed axes move by the selection rule: cheapest containing value, fewest new mechanisms, narrowest containing scope, rungs before moves, scope exclusion before hardening. Then the recovery axis is decided per effectful operation from the domain-shape facts: inverse exists and consistency must be restored — compensate; degradation tolerable and boundable — degrade and continue; the operation reshapes so the failure cannot occur: design it out, and check this option first.

A word on order, because the walk visits axes one after another and a reader is right to ask whether a different order lands elsewhere. It does not, and the reason is Chapter 3’s own definition: selection is *the cheapest vector whose capability envelope contains the demands* — a property of the whole vector, not a sequence of local moves. Which axes move, and toward which values, is fixed before resolution begins: pressures are recorded in step 3, from the demands against the starting vector, before any axis resolves — so no axis’s resolution creates or removes another’s pressure. What remains is choosing values, and *fewest new mechanisms* is counted over the finished vector, not tallied as the walk proceeds — a bus introduced for one axis is free for every other that can ride it, whichever order they were written down. The walk is the efficient route to a minimum that does not depend on the route; with six axes, a genuinely close case is small enough to settle by enumeration.

Step 5 — verify. The derived vector faces the exit gate — the next chapter — where every guarantee must name its mechanism and every budget must survive arithmetic. Verification is part of the procedure, and treating it as a later phase is how wrong vectors reach production with their paperwork in order.

The conflict rule

Step 4 contains the procedure's one genuinely subtle move: what happens when demands press one axis in *opposing* directions. The order platform above is the common case, and the rule handles all cases with one test.

Different scopes → split. When the opposing pressures attach to different paths, data classes, or subsystems, the axis refuses to compromise — a middle value would underserve both demands while paying for neither cleanly. The system splits at the boundary between the pressures, each part derives independently, and the split pays the boundary cost from Chapter 3 in the open. Checkout gets its transactional store; analytics gets its cheap reads; the boundary between them gets a contract and an operational seam, priced and visible. Note what the split is *not*: a decomposition by org chart, by noun, or by fashion. The boundary falls where the pressures diverge, which is the only place it earns its cost.

Same scope → decompose further. When opposing pressures attach to the *same* scope, the conflict is usually a bundle that step 1 under-decomposed. The canonical case from Chapter 2: team ownership pressing toward many deployables, operational cost pressing toward one. Decomposed, ownership turns out to be satisfiable by module boundaries at zero deployment cost, and the conflict evaporates: a modular monolith serves the binding parts of both demands minimally. Same-scope conflicts are sent back through decomposition, and most of them do not come back. The loop terminates, and the floor is defined: decomposition ends at *mechanism level* — the entry gate's own language — because a demand stated as a mechanism requirement has nothing smaller to split into. A mechanism-level pair still opposing at one scope has nowhere left to go, which is the next case.

Still opposed after decomposition → contradiction. When demands that genuinely share a scope genuinely oppose after honest decomposition, the derivation halts and says so — and this is the procedure working, not failing. The answers contain a conflict the business has not resolved, and no arrangement of software resolves a business contradiction; it can only hide one, expensively. What a useful method produces at this point is Chapter 8's subject: the contradiction, detected mechanically, priced as a menu of which answer could bend and at what cost.

The pressure matrix

The derivation's working artifact is a table: one row per demand (and per domain-shape fact), columns for scope, mode, shape, the axis pressed, the value demanded, and the mechanism that justifies the pressure — or the single word *inert*. Part II fills several of these tables in earnest; the appendix worksheet carries the blank.

The matrix does two jobs. During derivation it is the workspace. Afterward it is the *traceability record*: every position in the final vector cites the rows that forced it, which means the architecture can answer questions about itself — why this axis moved, what would have to change for it to move back. Chapter 9 builds the audit on exactly this property, and the decision records that survive into the repository are these rows in prose.

One thing the matrix must never be: pre-filled. A matrix shipped with answers already pressing axes is a catalog with extra steps — this book's own warning label. The instrument is the *empty* table plus the discipline for filling it; every filled version belongs to one system on one date.

What the procedure refuses to decide

Honesty about the mechanics requires naming their edge, and the edge is principled rather than apologetic. The procedure decides nothing about the answers themselves: targets are the business's to set; the derivation owns consequences. It does not choose within genuine recovery ties: where the domain offers both an inverse and tolerable degradation, the ledger prices both and a human weighs restored consistency against liveness, a business preference wearing a technical costume. It does not resolve contradictions — it detects them and hands back a priced menu. And it does not pick products: the vector says *distributed shared store*, and whether that is one vendor or another is a later, different decision — one the vector finally makes tractable, since candidates now face a specification rather than a beauty contest.

Everything else — every "it depends" that used to end meetings, every style debate, every seniority tiebreak — is inside the mechanics. That was the design goal: mechanize the steps that never deserved judgment, so that judgment concentrates where it is genuinely owed.

What the procedure is, mathematically

Stated in another discipline's vocabulary, `next_step` is **constrained optimization over a finite design space**. The answer sheet supplies the constraints. The ledger defines the space — six axes, a handful of atomic values each, composable by scope splits. The objective is cost; the output is the cheapest vector whose capability envelope contains the demands. A reader trained in operations research will recognize the shape at a glance, and the recognition is correct — nothing in the word *derivation* denies it. Naming it buys three boundary statements that would otherwise stay implicit.

The space is chosen, not complete. The procedure selects among mechanisms the ledger prices; it cannot invent one. When the field produces a genuinely new mechanism — a storage class, a substrate, a containment trick like the coalescer that opened Chapter 3 — it enters as a ledger amendment: a new value or rung with its own provides/mechanism/costs entry, available to every derivation after that day and invisible to every one before. What the procedure guarantees is minimality *within the ledger it ran against* — a checkable claim — not minimality over all possible engineering, which no method can promise honestly.

The objective is not a scalar. Costs arrive in currencies that do not convert — money, operating attention, latency floors, blast radius — and the selection rule already refuses the conversion: comparison is by mechanism count precisely because prices resist arithmetic. Where counting fails to settle it — two vectors, each cheaper in a currency the business must weigh — the procedure has found a genuine tie, and ties sit on the refusals list above by design. Weighing one currency against another is

business preference; a formula that hid the weighing would be the fake precision this book exists to refuse.

The minimum is global over that space. The order argument earlier in this chapter carries the weight — pressures fixed before any axis resolves, mechanisms counted over the finished vector, enumeration as the tiebreak that six axes keep cheap. A greedy walk trapped in local minima is what the procedure was built not to be.

So the architecture does not follow from reality itself. It follows from the answers, over the ledger, up to the ties that come back priced — and each qualifier is visible on the page: the answers sit on the sheet with owners, the ledger sits in the appendix with costs, the ties return as menus. *Derivation* names what the optimization vocabulary leaves out: where the constraints come from. They are elicited facts rather than preferences, which is why two teams holding the same sheet and the same ledger reach the same vector, up to the ties the procedure refuses. Constrained optimization is what runs; derivation is why the result binds.

What the procedure hands over is not yet a verdict. Every position in the vector is a claim, and claims get checked — the last instrument in the toolkit exists to do exactly that, before anything gets built.

Instruments introduced: `next_step` (null vector → normalize → prune → press → resolve → verify) · the regime insight · combination pressure · the conflict rule and scope test · the contradiction halt · the pressure matrix · the procedure's refusals · the optimization naming and its three boundaries.

Verification

A contract lands: quotes served in 200 milliseconds at P99, in-region, in every region the platform sells. The team celebrates the deal and files the number as a requirement. Nobody runs the arithmetic that would have taken ninety seconds: the platform masters its data in one region, a cross-region round trip costs 80 to 150 milliseconds of geography before any software runs, and P99 includes the bad days. Half to three-quarters of the entire budget is spent on the speed of light — at the *median*. The number was signed as a business term and it is an architecture verdict: either the data is served from every region where the promise was sold, or the promise is broken on the day it is tested. There is no third state, and no amount of engineering effort inside the wrong architecture reaches one.

A derived vector is a set of claims — this guarantee, from this mechanism, inside this budget — and Part I closes with the instrument that checks them. Verification is the exit gate of every derivation, and it earns its keep twice: it catches wrong vectors before they are built, and it catches fake answers that survived the entry gate, because a fake answer is often easiest to expose by pricing what it would take to honor it.

The consistency lens

The first check is qualitative and per operation: for every operation that matters, state the **guarantee** it holds under the derived vector, the **mechanism** that earns the guarantee, and the **behavior under failure**. Three columns, no exceptions, and the discipline is entirely in the second column: **a guarantee with no mechanism is a lie the system will eventually tell at the worst moment.** “Bookings are strictly consistent” — through what? Single-store transactions: earned. “Read-your-writes for the seller’s own view” — through what? If the reads land on lagging replicas and nobody can name the session-pinning mechanism, the vector is wrong or the answer was decoration.

The lens also enforces a vocabulary rule on the system’s own documentation: one-bit labels are banned outputs. A system is never “strongly consistent” or “highly available” in its own description — a specific operation holds a specific guarantee through a specific mechanism and degrades in a specific way. Systems described at one bit get defended at one bit, and one-bit defenses are how outages get explained to customers after the fact rather than to engineers before.

The failure column does quiet work that deserves notice: it is where the recovery axis’s choices become visible per operation. An operation whose failure behavior cannot be stated has not chosen a recovery class; it has chosen surprise.

Budget arithmetic

The second check is quantitative, and five rules cover what a derivation needs. None requires more than arithmetic — four of them not even that much — and all five are routinely skipped.

Rule 1 — latency budgets decompose downward. An operation's budget is spent by its critical path: sequential steps add, parallel branches cost their maximum, every network hop pays a round-trip floor, every cross-region hop pays geography.

path floor = Σ sequential floors + max(parallel branch floors) · software budget = target − path floor

The verification move is subtraction, and the chapter's opening contract is the whole worked example: target 200 ms; cross-region geography 80–150 ms; software budget 50–120 ms — for every hop, queue, store lookup, and serialization on the path, at the *median*, with P99 still owed the bad days. Those are the ninety seconds nobody ran. When target divided by floor approaches one, the select-mode driver from Chapter 2 is pressing — and when the floors alone exceed the target, the vector is wrong regardless of implementation quality. Physics decides whether the number is reachable; production only proves whether you reached it.

Rule 2 — tails compose through the distribution. The instinct is that a chain of five steps, each at P99 100 milliseconds, gives a chain P99 of 500. The truth is worse in the direction nobody budgets for: sequentially, the chain is slow when *any* step is slow.

series: slow-fractions add, P(chain slow) ≈ $p_1 + p_2 + \ldots + p_n$, so a chain of n steps held to allowance p grants each step ≈ p/n · wait-for-all fan-out over n shards: P(at least one past its P99) = $1 - (1 - p)^n$

Five steps each allowed to be slow 1% of the time produce a chain slow about 5% of the time — holding the five-step chain at P99 grants each step roughly a 0.2% allowance, a fifth of what its own P99 promised. Tail budgets divide like error budgets; means add like costs. Fan-out is crueler: a hundred shards at per-shard P99 of 10 milliseconds means $1 - 0.99^{100} \approx$ **63% of requests wait on at least one shard beyond its P99**. Fan-out harvests the tail rather than diluting it: the arithmetic behind request hedging, behind the coalescing rung, and behind restraint in partition counts. One caveat travels with both formulas: they assume independent slowness, and production correlates it — a shared garbage collector, a shared host, a deploy wave slow everything together. Correlation makes the sequential arithmetic kinder and the fan-out numbers optimistic; treat the independent-case results as bounds and budget for correlation the way rule 3 does explicitly.

Rule 3 — envelopes compose upward, by correlation. A subsystem's capacity envelope derives from its operations': steady loads add; peaks add only when correlated.

envelope = Σ steady loads + Σ peaks that fire together · anti-correlated peaks contribute their max, not their sum

The on-sale burst hits quote, hold, and payment together — one correlated peak, one envelope. Reporting close and pay-run may anti-correlate by calendar, and sizing for their sum wastes the difference. The calendar is capacity input, and shape beats headline everywhere: Part II contains a burst of a hundred thousand attempts per minute whose honest envelope decomposes into a large cheap stateless edge, a tiny contended core sized to *seat throughput*, and a fan-out absorbed by a buffer — three different envelopes from one headline number, none of them the naive division.

Rule 4 — availability multiplies down a chain; independence must be earned.

series: availabilities multiply, $A = a_1 \times a_2 \times \ldots \times a_n$ · parallel: unavailabilities multiply, $A = 1 - (1 - a_1)(1 - a_2)$, independence assumed

Series first: five components in a chain, each at 99.99%, compose to $0.9999^5 \approx 99.95\%$ — a 99.99% path cannot be built as a series of five parts as good as its target, and every thin tier in the chain counts in the multiplication even though none appears in the vector. The escape is parallel: two independent 99.9% paths compose to $1 - 0.001^2 = 99.9999\%$ — *if* the failures are independent, and shared deploys, shared certificates, shared regions, and shared configuration are the reasons the parallel arithmetic lies in practice. Earned independence is structural, which is the quantitative case for blast-radius isolation and for the cell escalation in Chapter 3: the arithmetic of redundancy is only as honest as the isolation underneath it.

Rule 5 — the mechanism bill must fit the operating envelope. The four rules above verify performance answers; this one verifies the bound-mode answers, which most methodologies elicit and never consult again.

Σ bills of standing mechanisms $\leq$ operating envelope, in both its currencies: money and operating attention

Count the derived vector's standing mechanisms — every store technology, every broker, every projection pipeline, every cell-management discipline — and price the count in the two currencies the envelope answer stated. The check is a comparison rather than a formula: a vector whose mechanism count assumes a platform team, held against a sheet that says four engineers and no operations staff, has failed verification as surely as a blown latency budget, before anything is built, at the cost of counting. Part II's chat-platform run is the worked case: the four-engineers answer pruned as hard as any latency number on that sheet, and a vector that ignored it would have passed every other rule and still died in production, slowly, of operational starvation.

The availability ladder is worth internalizing as price bands. Two nines and a half — 99.5% — is forty-four hours a year: a restart strategy with an on-call phone. Three nines is under nine hours: failure domains start mattering. Four nines is fifty-three minutes a year: series arithmetic starts vetoing designs, and the Chapter 2 pricing drill ("what does the business do differently in the fifty-third minute?") is what this chapter's numbers look like when they arrive at the elicitation table. The two gates are one discipline at two moments.

Verification before the build

The lens and the arithmetic check the vector on paper. Between paper and production sits a third pass, cheaper than either neighbor believes: exercise the claims before the system exists in full. Load models against the derived envelopes, with the correlation structure, not just the totals. Capacity models against the containment rungs' ceilings — the hardware rung's economics, a replica's lag under the projected write rate. And failure injection against the stated failure behaviors — every row of the consistency lens's third column is a testable hypothesis, and injecting the failure is the test. The industry practice this resembles is deliberate: game days and fault drills are the re-

covery axis's verification arm, and Part IV returns to where that practice's ownership belongs.

None of this replaces production truth. It bounds the surprises production is allowed to hold: a vector that survives decomposition, tail arithmetic, envelope composition, and injected failure can still disappoint — but at the margins, where operations lives, and no longer at the structure, where rewrites live.

The toolkit, complete

Part I is done, and it is small on purpose: a sheet of priced answers and domain-shape facts; a ledger of what values provide and cost; a procedure that walks one against the other and refuses three specific decisions; a gate that checks the output's claims. Everything fits in the appendix as a worksheet and a deck of reference cards, which is a design property rather than a convenience — a method you can apply from memory is a method that actually gets applied.

What a toolkit cannot do for itself is prove that it works. Part II exists for that: first the domain this book controls, re-derived with every step citing its rule — then four systems this book does not control, derived blind from their public commitments, with predictions registered in advance and graded in the open. The instruments just described are about to be held to their own standard.

Instruments introduced: the consistency lens (guarantee / mechanism / failure behavior) · the one-bit-label ban · the five rules of budget arithmetic · the availability price bands · pre-build verification (load, capacity, injection).

Three Profiles, One Domain

Part I ended with a promise of proof, and proof starts on home ground: one domain, event ticketing, at three scales — an independent venue, a regional platform, an enterprise multi-tenant platform. Readers of *Process-First Design* have met these three profiles in the Architecture Synthesis module, where their derivations were narrated; this chapter supersedes that walkthrough by re-running it with the mechanics visible — every step names the answer it consumes and the rule it applies. Same systems, same outcomes, and the judgment that the narration quietly exercised now sits in tables anyone can check.

The domain earns its place by spanning the extremes with one product idea. Selling a seat is selling a seat at every scale; what changes between the profiles is only the answer sheet — which is precisely the experimental condition Chapter 1's claim requires. If the architecture follows from the answers, three answer sheets over one domain must produce three different vectors, each forced, with no step appealing to taste. One honesty belongs at the door rather than the exit: these are the book's own systems, graded by their author — home ground proves the mechanics legible, not the method right, and the stricter test follows in the next chapter.

Profile one, promoted to notation

Chapter 1 already walked the venue in prose; its home telling stands, and here it is promoted into the derivation's working form — the pressure matrix rows the prose was narrating:

Demand (scoped)	Mode	Against the null vector	Outcome
buy ≈2s P95 (operation)	select	contained: direct calls + single store at tens/sec	inert
check <1s (operation)	select	contained	inert
99.5% (domain)	isolate	contained: single node + restart budget (44 h/yr)	inert
booking strict, zero loss (data class)	prune	single shared store provides transactions + durable commit	strikes nothing
reads eventual (data class)	—	weaker than what the vector already provides	inert

Demand (scoped)	Mode	Against the null vector	Outcome
cost ceiling, one team (domain)	bound	reinforces the null position	inert
read-moderate (domain)	split	no divergence between paths	inert

Zero pressures; the null vector survives; recovery resolves from domain shape (defined inverses on the money path → compensate). One derivation artifact deserves attention beyond the outcome: the *inert* column is doing the work. Every row that presses nothing is a documented refusal — the reason this system carries no queue, no replicas, no projections, written down where the next architect can find it. Chapter 9 turns exactly this property into an audit.

Profile two: the regional platform

The venue joined a network — Chapter 1 previewed the first consequence — and the full sheet at regional scale reads: several teams and venues, one region, multiple currencies and countries; availability checks at 500 ms P95 on a read-heavy path; purchases still ≈2 s; a sales feed consumed asynchronously by venues; bookings strict with zero loss; quotes tolerate bounded staleness, but a venue must see *its own* availability updates immediately; 99.9% service target; weekly blue-green deploys; payment-card compliance; event-heavy integration traffic.

The walk, each step citing answer and rule:

Deployment — the decomposition case. "Several teams need independence" presses toward multiple deployables; the operations budget presses against; both attach to the whole system: same scope, so the conflict rule sends the bundle back through decomposition rather than splitting anything. Decomposed: ownership boundaries are demanded — and module boundaries contain them at zero deployment cost; release-cadence independence is *not* demanded — one weekly blue-green train is the stated cadence for everyone. The conflict evaporates. **Modular monolith**, and the month of meetings this resolution replaces is the one Chapter 4 opened with.

Read serving — the rung case. 500 ms at P95 on a read-heavy path escapes what primary-only serving contains: pressure on read serving. The chain from Chapter 3 climbs rung by rung: caching helps but the reads are per-venue and varied; the next rung, **read replicas**, contains the volume, and the staleness the quotes tolerate is exactly what replicas cost. The top rung stays unclimbed: a separated read model would add projection machinery and buy nothing these answers demand, because the read *shape* has not diverged — this is volume, and the second-copy diagnostic says yes. The read/write model stays **unified**.

Read-your-writes — the mechanism case. The venue-sees-own-updates answer is a prune-mode fact replicas alone violate. The lens forces a mechanism: session-pinned reads to the primary for a venue's own recent writes. Contained without an axis move

— a mechanism note, not a value change, and the difference is the vector staying honest about what it is.

Substrate — mixed by scope. The sales feed and cross-module facts tolerate lag and want fan-out: the ledger's event-based entry, across module boundaries. The booking sequence wants strict immediacy within its module: direct. **Event-based across, direct within** — a composition, not a compromise, and each scope is uniform.

Storage — the trap, disarmed by comparison. Payment-card compliance plus dispute pressure sounds like "we need full history," and this is the audit-versus-replay decomposition's home game. Decomposed: auditability is demanded; replay is not — no answer on this sheet needs past states under past rules. Cheapest value containing audit alone: **current-state plus an audit log written in the same transaction**. Event sourcing would contain the demand too — at the price of projections on every read, forever, for a capability no row of this sheet cites. The narrated version of this walkthrough called this a trap; the mechanical version shows there was never a trap, only an under-decomposed answer meeting a ledger comparison.

Recovery. Money keeps its inverses (compensate); reservations and counters reshape — idempotent holds, converging counters — so their conflicts stop existing (**design-out** at that scope).

Derived vector: *modular monolith / event-based across + direct within / unified + replicas / current-state + audit log / single shared store + replicas / compensate + design-out.* Every position cites rows; the two "traps" of the narrated version fall out of ledger comparisons with no judgment anywhere.

Profile three: the enterprise platform

The sheet turns hostile: quotes at 200 ms P99, *contractual*, multi-region; on-sale bursts of 10^5+ attempts per minute against a handful of seats; price history auditable **and replayable** — a regulator reconstructs quotes under past rule versions; bookings strict, zero loss, across regions; availability reads with a named staleness bound; 99.99% on the customer-facing read path; data sovereignty by country; card compliance; weekly canary releases; cost under real scrutiny.

The split — the scope test's home game. The read path carries 200 ms P99 + 99.99% + storm-scale reads; the transactional cores carry saga consistency at moderate scale. Opposing pressures — and this time at *different* scopes: path versus subsystems. The scope test splits at the boundary: **the read path becomes its own component** with its own scaling and cadence, and blast-radius isolation falls out of the same split, which is what Chapter 5's rule 4 said honest 99.99% would require. The boundary cost is paid knowingly: a contract between quote-serving and the cores, a staleness bound already priced by the answers.

Pricing — the axis that finally moves. Replay *is* demanded, for one data class. **Event-sourced pricing**, with the quote storm served from **projections** — the read model separates *here*, where shape and volume and contract converge on one path, and nowhere else. Booking, by the same comparison as profile two, stays current-state with its audit log: two data classes, two storage answers, one system.

Booking across regions — the unique container. Strict + zero loss + multi-region is the demand set with exactly one containing value in the whole ledger: the **distributed shared store**, quorum commit, write floor paid where physics charges it. Every other persistence value is struck in the prune round — the derivation's only forced buy at the top of the market, and the sheet shows precisely which three answers forced it.

The cores — packaging held open. Strongly coupled subsystems, scale and deploy as units, ops cost bounded: the ledger's **unified runtime** entry, one-or-many packaging decided at deploy time, contains the bundle most cheaply (the runtime class Chapter 3 disclosed; the derivation needs the *value*, and any instance of it serves — the one selection of that value in this book, in a system its author controls; Chapter 7's four blind runs never demand it). Event management's document-shaped and relational-shaped data lands **polyglot**. Sovereignty pins data regionally *within* the distributed store — a constraint on placement, no further axis move.

The burst — contention, refused correctly. 10^5 attempts per minute against a handful of seats: the second-copy diagnostic says no copy helps — one seat, one winner. **Admission control at the edge, fast-fail, holds that decay** — the entire "storm" is contained with zero axis movement, by the envelope arithmetic Chapter 5 ran: a large cheap edge, a tiny core sized to seat throughput. The most dramatic number on the sheet moves nothing, and that is the discipline holding under maximum provocation.

Derived vector (per scope): *read path as services + cores on a unified runtime / event-based across, direct within / separated (pricing) + unified (booking) / event-sourced (pricing) + current-state (booking) / distributed shared (booking) + per-component + polyglot / compensate (money) + design-out (holds, pricing appends) + degrade-and-continue (availability views).* Vectors per subsystem — the composition Chapter 3 priced, arrived at by recursion rather than by decree.

What three sheets prove

One domain, three sheets, three vectors — none chosen. The venue proves the null vector is reachable in practice, and that restraint is an output. The regional platform proves conflicts decompose and rungs beat reflexes. The enterprise platform proves the expensive values are purchasable *when rows force them* (event sourcing at one data class, a distributed store for one data class, a split at one boundary) and that even a hundred-thousand-a-minute headline cannot move an axis without the right shape.

The objection stated at the door still stands at the exit: home ground, author-graded. Chapter 8 will show the method failing honestly, but before that, the stricter test — four systems this book had no hand in, derived from their operators' published commitments, with predictions registered before checking outcomes. Home ground is done.

Rules exercised: the null vector's survival · decomposition at the entry gate · the scope test (same-scope and different-scope branches) · the containment chain (replica rung, projection rung) · audit-vs-replay · the unique-container prune · contention refusal · per-scope recursion.

Systems Nobody Asked Us to Derive

A method validated only against its author's own examples has passed a take-home exam it also wrote. The remedy is systems this book had no hand in: derive them from their operators' *published commitments alone*, register predictions before checking outcomes, grade in the open, and keep the misses. Four systems take the exam here — Stack Overflow in its 2016 configuration, Shopify in the pod era, Discord's message path across 2017–2023, and a British government registrar whose place in this chapter needs its own explanation when its turn comes.

Honesty about the protocol first. True blindness is impossible for famous systems — their architectures are folklore. Defensibility comes from the same discipline the whole book runs on: **every derivation step cites a published answer, never the known outcome**, with sources and eras pinned so inputs and outcomes match in time. Predictions go on the record before the answer sheets are assembled, which creates the possibility this chapter values most: the registered prediction and the derivation can *disagree*, and when they do, one of them is about to be graded against reality. It happens twice below.

Because the four runs were not equally protected, their evidence carries explicit grades, worst protection first:

Grade	Protection	Runs
C — registered, contaminated-adjacent	prediction registered in advance, but drawn from memory of the same public sources it would be graded against	Discord (the registered revision)
B — registered, clean	prediction registered before the answer sheet was assembled; operator not isolated	Stack Overflow, Shopify
A — isolated operators	derivation executed by operators quarantined from the outcome	Companies House

Three more disclosures belong in the protocol note, because a careful reviewer will ask and the answers favor honesty over polish. *Verifiability:* the registrations for these four runs are internally documented (dated working files, session records) and attested here rather than externally provable; the artifact set — answer sheets, registered predictions, grading rubrics, and for the fourth run the derivation transcript and operator protocols verbatim — is published as a replication kit at the repository the references cite, and every run from the repository's creation onward registers in public, by commit, where timestamps verify themselves. *Selection:* all four systems are

famous publishers of engineering writing — organizations selected, by the method's own sourcing needs, for having coherent documentable architectures; the protocol requires published answers, and publication is not a random sample of the industry. *Scoring:* graded positions are not independent (a single-deployable position makes direct calls nearly automatic), so raw hit counts flatter any method, including this one. The honest baseline question is sharper and is answered per run below: **what would "always predict the boring monolith" have scored, and where did the derivation beat it?** Three of the four systems derive to mostly-null vectors, so the naive default scores respectably on position counts. It misses precisely the calls that matter: it predicts read replicas at Stack Overflow where the answers refuse them; it has no way to reach Shopify's full-stack cells or the edge admission that saves the flash sale; and it keeps Companies House's register reads unified where the statutes force the split. The method's value over the default is concentrated in exactly the positions where getting it wrong is expensive — which is what a selection procedure is *for*. The baseline a critic would rightly prefer is sharper still: not a naive default but an experienced architect's registered judgment on the same sheets. That comparison exists exactly once in this set — the author's own Stack Overflow prediction, registered and then beaten by the derivation — and once is not a corpus; running it properly, with architects who are not this book's author, is standing work the replication kit invites.

Stack Overflow, 2016: the answers beat the prior

The published sheet, from the operators' own engineering posts: ~209 million HTTP requests a day against a 50-millisecond render budget; a store-level read/write ratio of 40:60 — *write-majority at the database* under a read-dominated HTTP tier; the entire database cached in memory; a relaxed availability stance stated outright, with deploys taking servers out routinely; ~15 committers deploying 5–10 times a day as one train; a loud cost stance — "hardware is cheaper than developers and efficient code"; no compliance, audit, or replay statement anywhere; and two workloads with genuinely divergent shapes: tag matching (a giant in-memory index with a two-minute reload cycle) and full-text search (inverted-index access, with a published cost citation for offloading it).

The registered prediction: a monolith with **read replicas**, plus two scope splits. The derivation, run against the sheet, refused half of it.

Deployment: fifteen committers on one uniform train, ops-cost ceiling explicit, availability stance tolerant — nothing presses; single deployable, N identical instances as capacity. **Persistence — the step that killed the prediction:** the reflex says read-heavy site, read replicas. The answers say the store is 40:60 write-majority and the whole database lives in RAM: the primary *contains* the read demand outright. What the sheet does press is redundancy: failover drills, a disaster-recovery site. Cheapest containing mechanism: an HA replica that serves failover rather than traffic, asynchronous because the accepted loss budget permits it. **The hardware rung, as company policy:** the 50-millisecond budget at three thousand requests a second is contained with zero axis moves because they sized the null position: RAM for the working set, web tier idling at 5–15% CPU. Chapter 3's rung zero, operated as an explicit value statement — and the engineering culture Chapter 3 left unnamed is this one.

Two splits, exactly: tag matching's shape (divergent state, divergent lifecycle) and search's shape (divergent access, plus their own cost citation) each escape the web tier's containment — two components, at those two scopes, nothing else diverging. **Unified reads** with anonymous-scope caching (the cache rung, one below replicas). **Current-state** storage, roll-forward operational posture, direct calls with one scoped pub/sub for cache invalidation.

Graded against the documented 2016 architecture: a .NET monolith on nine web servers; a tag engine on three dedicated servers and an Elasticsearch cluster — *two splits, those two*; SQL clusters described by their own operator as one primary taking almost all load with an AlwaysOn replica — **the replica is an availability mechanism, exactly as derived and exactly against the registered prediction**; Redis caching with no separated read models; roll-forward migrations. Every axis matches, including the splits' count and location, and including the detail the prediction got wrong.

That miss is the finding. The prediction was a taste-shaped prior — *read-heavy site, therefore read replicas* — and the derivation, forced to cite answers, refused the taste and was right. The thesis ate its own author, first, in public, and the lesson generalizes: the reflexes this method replaces include the author's.

Shopify, pod era: the expensive values, earned

The sheet: 80 thousand requests a second at peak across 600 thousand merchants, growing to holiday peaks of 11 million queries a second and 11 terabytes a second of read I/O; flash sales described by the operator as Black-Friday-scale "every seven or eight days," selling out in seconds; a 2016 incident stated as a demand — one celebrity sale took down every store sharing the database shard; merchants isolated from each other with no cross-shop actions; over a thousand developers, 400 commits a day, deployed as *one train* around forty times a day; payment-card compliance handled, in their words, by never letting the monolith see card data; checkout write-concentrated on single records, storefront read-dominant and staleness-tolerant.

The derivation's spine: read volume past any single store's economics, with a **natural partition key** the multi-X question hands over (shops never interact), gives *sharded on the shop*. The 2016 incident plus per-merchant stakes demand that one tenant's burst cannot reach another; a store-only shard leaves shared caches and queues as cross-tenant failure paths, so the shard boundary **widens to the full stack**: cells, Chapter 3's escalation, derived rather than defined. A thousand developers press ownership; the uniform train presses against fragmentation; same scope, decompose. Ownership is real, release independence is explicitly not wanted: **modular monolith**, the same resolution as the regional ticketing platform at twenty times the headcount. Flash-sale contention, every checkout step hitting the same records, is the second-copy diagnostic saying no: **admission control at the edge**, queueing and fair ordering before the app tier. Storefront reads at 11 TB/s escape the primary (unlike Stack Overflow's RAM-contained reads, same chain, different citations, different rung): **cache plus replicas at the storefront scope**; checkout stays unified and strict. Card compliance: **scope exclusion** — split the card path out; the monolith exits the mandate's reach entirely.

Oversell: reservation during payment — a hold, **design-out**. Money: defined inverses, compensation.

Graded: "We chose to evolve Shopify into a modular monolith" — their words; pods as "a fully isolated instance of Shopify with its own datastores," no actions across pods, a hundred-plus of them; a leaky-bucket edge throttle with queue pages and fair ordering; storefront reading from dedicated replicas with full-page caching; the monolith kept out of card scope. Every registered prediction graded a hit, one under-prediction noted: the derivation's *design-out via reservation* was sharper than the registered "compensation for checkout money."

Discord, 2017–2023: the revision that beat the fashion

The sheet: forty million messages a day growing to four billion, trillions stored under a product commitment made early and kept — "store all chat history forever"; reads "extremely random" at a 50/50 store ratio, drifting read-heavy at the disk; sub-millisecond write and five-millisecond read ambitions with an 80-millisecond P95 alert; five million concurrent connections fanning out millions of events a second; availability preferred explicitly over strong consistency, last-write-wins, per-channel ordering by sortable IDs; hot channels concentrating "unbounded concurrency" on single partitions; and the loudest bound-mode answer in this chapter — **four backend engineers, no dedicated operations team**.

Two predictions were registered, one of them against this book's own earlier claim. Queuing Discord for derivation, this author had described its read path as "genuine read-model separation." The registered revision said that prior was wrong — expect **a coalescing layer over a unified store, not CQRS**.

The derivation: trillions of rows, node-loss tolerance, linear scaling, four engineers — the single store fails on volume and on operations at once; the partition key is again a gift (messages belong to one channel, reads are per-channel recency), **sharded on (channel, time bucket)**, in a **wide-column store shaped to the read** (polyglot at the message scope). The real-time fan-out workload diverges from request/response on every dimension: **one split, the gateway tier**; the API itself stays a monolith — four engineers press hard against anything more. Hot partitions are read-side *contention*, one channel has one home, so more sharding cannot help: **coalesce identical in-flight reads, bound the concurrency**; the read model stays unified. Writes reshape (upserts, last-write-wins, sortable IDs), so write conflicts stop existing: **design-out**. Presence is ephemeral and re-derivable: **degrade-and-continue**, decay as a feature.

Graded: an Elixir gateway tier beside what the operator calls "our API monolith"; the exact key — `((channel_id, bucket), message_id)`; Rust data services with "no business logic," one endpoint per query, **coalescing routed by channel ID** over the store — the registered revision graded a hit, the fashion (call it CQRS and separate the reads) graded as what the answers never demanded. An honesty note stands in the record: that revision drew on memory of the same public sources, so it counts as a registered-and-confirmed call, a weaker species of evidence than Stack Overflow's clean answers-beat-prior.

One more Discord finding pays for the whole exercise. Their 2017 store migration (document store to wide-column) *moved an axis* — architecture-grade, hard, slow. Their 2022 migration (one wide-column product to another) moved **no axis** — same position, different product. The vector classifies migrations into architectural and procurement before they start, and the two migrations' documented difficulty matches the classification. Part III builds on exactly this.

Companies House: the boring one, and the strictest protocol

Three consumer-scale celebrities prove range but leave two gaps. The method's home turf (enterprise backends: statutory deadlines, compliance, moderate scale, long lives) was unrepresented. And one axis had never moved: across three blind runs, the read-/write model stayed unified every time, raising a real question of whether the axis deserved its seat at all or should fold into storage. The fourth run was chosen to close both gaps: the UK's company registrar — 5.4 million companies, statutory filings, a public register — dull by design, and instrumented to be decisive about the read axis either way.

The protocol also tightened, for an honest reason: during target scouting, outcome details leaked into the orchestrating context: research summaries described the real architecture before any derivation ran. The author was contaminated, and rather than pretend otherwise, the run was restructured so contamination could not matter: **three isolated operators** — an assembler that gathered the answer sheet from demand-side sources only (statutes, audit-office reports, annual accounts: technology venues forbidden); a deriver that received *only* the answer sheet and Part I's instruments, with no ability to browse and a standing quarantine rule (any conclusion untraceable to a cited answer plus a named rule is forbidden); and a grader that assembled the documented architecture independently, with citations, after the derivation was registered. One disclosure completes the methods note, foregrounded rather than buried: **the three operators were AI agents**, each a language-model instance executing its written protocol, with the prompts preserved for the replication kit. Far from weakening the run, this is the mechanizability thesis demonstrated at its strongest — a derivation procedural enough that a quarantined machine can execute it from the sheet and the rulebook alone is a derivation that has genuinely left the realm of taste; the operator could not *browse* to the outcome — and, a machine executing a written protocol, had no stake to defend and no prediction to protect — while every conclusion it drew traces to a cited answer and a named rule. One limit stays on the record: Companies House is a well-documented public institution whose engineering writing plausibly sits in any modern model's training corpus, so the protocol controlled *contextual* contamination, not *parametric* memory — procedural blindness, not informational blindness, which no language-model operator can currently promise. The evidence that the two came apart here is the miss itself: the derivation predicted the forced minimum and the real registrar had decomposed further; whatever the model had absorbed, it did not quietly copy the documented answer. A run steered by memory of the outcome fails *toward* the outcome; this one failed away from it. A second distinction the framing owes the reader: the target's *selection* was not blind — the run was aimed, deliberately, at a

system likely to be decisive about the read axis. The strong claim is about how the derivation ran, not how its subject was chosen; that a statutory registrar reads heavier than it writes surprises nobody, and the surprise the run earns is the axis moving only under cited convergence, not the direction it moved. The methods note stays in the record because that is what methods notes are for.

The demand side is an elicitation showcase. The register was accessed 16.3 billion times in a year against 14.7 million filings: a thousand-to-one read/write asymmetry, from the annual report. December concentrates an accounts-filing peak (28,874 companies once missed the deadline outright; 503 filed in the final hour). And the statutes answer the consistency contract better than most product owners do: a filing accepted in error is *not edited*: a correcting filing layers on top, the original remaining "properly delivered"; true removal takes a court order; a restored company "is deemed to have continued in existence" — a retroactive legal fiction the storage design must honor; and by statute certain fields (a director's day of birth) must be *withheld* from public inspection while remaining on the record — the served shape differs from the stored shape, by law.

The derived vector's spine: a modest modular core (release structure UNKNOWN, criticality tiers UNKNOWN, and honest UNKNOWNs derive null positions with low confidence, per the method); event-based filing intake (queued, deadline-shaped); **the read/write axis moved** — public search separated from the filing path, and the forcing was *cross-question convergence*: the thousand-to-one ratio (a volume answer) alone would have stopped at replicas; the statutory redaction duty (an external-constraint answer) alone is a mechanism note; together they clear the bar neither clears alone — the read *shape* diverges from the stored truth by law, at register scale. Current-state storage with corrections as layered records (the statutes' own append semantics); no event sourcing: nothing demands replay; the registrar's new query-and-annotate powers derive as flagged-pending states; recovery mixed: design-out for the append-shaped register, degrade-and-continue for backfills gated on each company's own filing anniversary, compensation only where courts order removal.

Graded — ten hits, two partials, two misses, nothing ungradeable. The hits include documentary confirmations of almost embarrassing precision: the public API schema carries a literal `annotations` field on filing-history items; the operator's engineering blog describes the pipeline that keeps a search index current from the filing-side stores by name. **The read-path prediction graded a hit against first-party documentation — the axis moved in a blind run for the first time, and reality confirmed the move.** The watch item died; the axis kept its seat by earning it.

The instructive miss is topology. The deriver predicted a modest modular core and flagged that prediction, in advance, as its most exposed claim: resting on a rule from the first three runs ("headcount presses cadence, never topology") plus three UNKNOWNs staying inert. Reality: the registrar's engineering organization describes itself as having adopted "the microservice pattern," with several hundred public repositories of fine-grained services. The grader suggested the answer sheet should have included that self-description; the suggestion is declined on principle: *"we adopted microservices" is the outcome, and feeding it to the deriver makes the experiment circular*. The legitimate gap was the UNKNOWN release-structure answer. And the

miss rescopes the rule rather than breaking it: **nothing in the published demands forces the decomposition** — the rule speaks about what forces topology, and it held; what it cannot do is predict what organizations *choose* beyond the forced minimum. A blind derivation predicts the demanded architecture; the gap between that and the observed one is either an unpublished demand or an unforced position — and detecting unforced positions is, from Chapter 9's angle, exactly what the audit is *for*. The method under-predicts fashion. It is supposed to.

To keep that rescoping honest, name what would break the rule rather than bend it. One case would: topology fragmentation with no forcing demand *and* no organizational home — no separate teams, no divergent cadence, no blast-radius line — structure the unforced-position ledger cannot file anywhere. And one trend would, more decisively: if blind runs keep under-predicting topology — if the forced minimum sits systematically below what disciplined organizations build — then something real is pressing that the sheet does not carry, and "unforced" has become this method's word for its own blind spot. One run does not make that trend; the commitment is that the ledger of misses stays open to both.

Cross-findings

Four runs, one instrument panel, and the patterns that recur are the ledger validating itself in the field. **Contention is side-symmetric:** write-side admission control at Shopify's flash sales, read-side coalescing at Discord's hot channels (the chapter's C-grade run, discounted accordingly): the same refusal to shard a one-home problem, at two of the industry's most-watched systems. **The containment chain stops where the citations stop:** RAM at Stack Overflow, replicas at Shopify's storefront, coalescing at Discord, projections at Companies House: four different rungs, each forced by that system's answers, none by fashion in either direction. **Team size never pressed topology:** four engineers and a thousand-plus both derived to monoliths from opposite ends, and the one microservices outcome in the set arrived *unforced by any published demand* — the strongest public evidence this book can offer that "we're big now, so services" is not a derivation. **Scale shape is per-tier:** the same system graded read-heavy at HTTP and write-majority at the store; every bare adjective would have been wrong twice. And **product identity presses like any SLO:** "store all chat history forever" was voluntary — and it pressed the persistence axis as hard as any regulator, priced in the operational record of the company that said it.

Two misses in four runs, both kept, both converted into rules — one the author's registered prior, which the derivation overrode; one the derivation's own, the boundary between forced and chosen. A method that can say *this is where I stop predicting* has a shape; the next chapter shows the sharper version of that property — the derivation refusing to produce an architecture at all, and what it hands back instead.

Rules exercised: citation-or-nothing · pre-registration and open grading · the hardware rung as policy · natural partition keys from multi-X · cells from compounded demands · scope exclusion · the coalescing rung · cross-question convergence · UNKNOWN-derives-null · the forced/chosen boundary.

When the Derivation Says No

Every validation so far succeeded, which should make a careful reader suspicious. A method that always returns a confident answer is hiding the cases where it has none. And for architecture, those cases exist, because some answer sheets describe systems that cannot be built. What distinguishes an honest procedure is what it does then: detect the impossibility *mechanically*, refuse to paper over it, and hand back something the business can act on. This chapter walks the refusal end to end. The case is constructed rather than sourced — flagged as such, and standing until a reader's real system replaces it, an invitation the closing chapter makes formal. Two real specimens already bracket it from either side: statute supplies a standing contradiction pair (a replay mandate and an erasure mandate landing on one data class — Chapter 2's three-way decomposition names it, Chapter 3 prices its containments, and where neither containment applies, the collision below is exactly what it looks like); and Chapter 11 contains a field specimen at national scale, a contradiction faced by history rather than constructed for a book. The clean room comes first because every part of it can be inspected.

Three answers, each reasonable

A trading platform for a commodity with a single global market. The answer sheet passes the entry gate — every answer priced, owned, and confirmed:

- **A1, compliance (prune-mode):** regulators in three jurisdictions require resident traders' order data stored and processed in-country. Statutory; sovereignty pins by law.
- **A2, consistency (prune-mode):** the business requires one global order book with atomic matching — every order sees every other order. Decomposition was attempted at the gate, the way it dissolved "team independence" in earlier chapters: is "one book" really global price *visibility*, with bounded staleness tolerable? No — two matching engines would create arbitrage between them, which the market's own rules prohibit. The demand survives decomposition intact, which matters below.
- **A3, latency (select-mode, contractual):** order acknowledgment within 50 milliseconds at P99, in-region, in all three regions.

Each answer, alone, is ordinary. Two at a time, they are manageable. All three define this chapter.

The walk into the wall

Prune round, persistence axis. A2's atomic global matching demands a single serialization point for the book. The candidates, against the ledger:

- *Single shared store, one region:* processes two jurisdictions' orders out-of-country (struck by A1) and serves two regions' acknowledgments across geography that alone exceeds 50 ms — struck by A3. Struck twice.
- *Sharded, by region or by instrument:* regional shards are two books — arbitrage, struck by A2. Instrument shards change nothing — the book is per-market and the market is one. Struck.
- *Distributed shared store, multi-region quorum:* satisfies A1 (regional presence) and A2 (one logical book). Against A3, the arithmetic from Chapter 5: quorum commit across three regions carries a physics floor of inter-region round trips — 80 to 150 milliseconds before any software runs, against a 50-millisecond promise. Unreachable at any engineering effort, at any budget, from any vendor. Struck by geography.

The axis is empty. Every value is struck. This is a state the procedure recognizes rather than argues with.

Scope test. Opposing pressures resolved by scope in every earlier chapter — so, different scopes? No. A1, A2, and A3 all attach to the *same* data class (the order book) and the same operation (submit, match, acknowledge). There is no boundary between the pressures to split along. The escape hatch that produced the enterprise platform's read-path split does not exist here — and the test proving its absence is the same test that proved its presence there.

Re-decomposition. A2 already survived the gate. A3 decomposes along exactly one soft joint, noted for later: does "acknowledgment" mean *matched*, or *durably accepted*?

Verdict: contradiction. No vector satisfies the three answers. The derivation halts and says so — and every element of the detection was mechanical: an axis emptied by pruning, a scope test with no boundary to find, demands that hold their shape under decomposition. No judgment participated in detecting the impossibility. Judgment is about to get the only job it can do well here.

The output: a priced menu

Halting with "impossible" would be true and useless. The derivation's real product at a contradiction is the explicit menu of which answer could bend, each option priced — the moment where *methodology informs, business decides* stops being a slogan and becomes a document:

Bend A3 — redefine acknowledgment. Ack means *durably accepted in-region*: a local durable enqueue, honestly achievable within 50 ms; matching completes asynchronously, with a match notification following within a few hundred milliseconds. The consistency lens states the new guarantee exactly: 50 ms covers acceptance; the match itself remains strict, global, and slower. Price: the semantics are visible to traders — fast acknowledgment stops meaning fast fill. This branch is where most real exchanges live, which is worth exactly as much as evidence usually is.

Bend A2 — regional books. Three books, with cross-region arbitrage explicitly permitted and bounded: publish the inter-book spreads and let arbitrageurs close them. Price: the market's one-price rule is gone, and the regulator who prohibits arbitrage

must agree. A bend obeys the same scope discipline as a value — applied at the narrowest scope that relieves the contradiction — and this is where the gate's finding that A2 survived decomposition intact collects its due: one market, one instrument class, no narrower scope to cut, so the bend is total or nothing. This branch usually dies in the room — and a branch that dies in the room is information the business paid nothing to learn.

Bend A1 — move the law, not the software. One jurisdiction hosts the matching engine under a treaty or equivalence arrangement; residency is satisfied by regional replicas-of-record. Price: years, lawyers, and a standing dependency on regulation holding still — an answer with its own change driver attached, which Part III would track like any other.

Reject all three. Do not build this product across three jurisdictions. A valid business answer, and vastly cheaper discovered at a whiteboard than discovered in production with contracts signed.

Two properties of the menu carry the chapter. First, the winning branch was found by decomposing the *softest demand* (the one joint that gave under pressure) rather than by out-shouting the loudest stakeholder; the procedure located the give mechanically, in step 3 of the walk. Second, a chosen branch is not a consolation prize: bend A3 and the new answer set re-enters the derivation and completes without incident — regional durable intake, one strict matching core, asynchronous match feeds. Failure is a fork, and both tines are ordinary derivations.

The other ways a derivation stops

The emptied axis is the sharpest halt; four duller ones round out the honest inventory.

Infeasible intermediate. The target vector derives cleanly and no operable path leads there from where the system stands — every route transits a state that cannot run. The resolution is a *scaffolding step*: a deliberate interim position, priced to include its own demolition. This is Part III's territory, named here because it is a failure of the *path*, never of the target.

The trapped state. The audit shows the current position was never forced, and the exit costs more than the occupancy: unwinding a wrong event-sourcing decision after three years of accumulated events is a real bill the menu must state, even when every option on it is bad. The derivation's honesty here is refusing to pretend the sunk mechanism is free to keep *or* free to leave.

The knowledge gap. Answers come back UNKNOWN on rows the derivation cannot proceed without — a consistency contract nobody will commit to, a loss budget nobody owns. The output is the *list of blocking unknowns*, which converts an architecture stall into an elicitation task with names attached. Chapter 7's fourth run modeled the benign version: UNKNOWNs derive null positions at low confidence, and one of them became the run's instructive miss.

Unexplored territory. The answers are real and priced, and the ledger has no entry that contains them — a demand shape without a named mechanism family, a mechanism class with no operational record to price. The gap is in the *instrument*, and the

honest output says so: “the ledger cannot price this” is a different statement from “this cannot be built,” and confusing them in either direction is how methods overreach. This halt is where the ledger grows (a new entry earns its way in exactly as the axes and questions did, by demonstrated pressure) and it is the halt this book most expects its readers to trigger.

A method’s character shows at its boundaries. This one halts loudly, prices the exits, and resumes cleanly — and having now seen it work and seen it stop, the remaining question is time: what happens to a derived architecture after it ships, when the answers start moving under it. Part III is about systems that live.

Rules exercised: mechanical contradiction detection (emptied axis + no scope boundary + decomposition-stable demands) · the physics floor as a pruning agent · the renegotiation menu · softest-demand decomposition · re-entry after the fork · the halt inventory (scaffolding, trapped state, blocking unknowns, unexplored territory).

Architecture as a Derivative

Chapter 1 planted a definition and deferred it: an architecture is a *derivative* — the next correct step from where the system stands, given the forces acting on it now. Part II ran that computation from blank pages. Part III runs it where almost all real engineering happens: on systems that exist, carry history, and keep living while the answers move underneath them. This chapter establishes the calculus; the two after it handle the walking and the inheriting.

The reframe is one substitution. `next_step` never required an empty starting point — step 0 said *start from where the system stands*, and greenfield was merely the case where it stands nowhere. Feed the procedure a real system's current vector and its current answers, and three distinct operations fall out of one function, depending on what moved since last time.

Nothing moved: the audit

Run the derivation with unchanged answers against the current vector, and the output should be *no change*. That sentence sounds vacuous and is the opposite: it is a mechanical test that every position the system holds is still forced by a live answer — and it fails in a specific, useful way. Any position the answers cannot force is flagged, and the flag has a name this book has been building toward since Chapter 1: **a position no answer forces is technical debt — by construction, not by opinion.** You are paying an always-on cost, per the ledger, for a capability no row of the sheet demands. One pre-commitment keeps the label honest, of the same kind the blind runs made for the topology rule: if audits keep surfacing unforced positions that demonstrably earn their cost — option value exercised, not merely claimed — the finding is a missing question, not debt, and the sheet grows the way the ledger does. A label that names its own disconfirmation is entitled to be mechanical; one that cannot be wrong is just vocabulary.

This flips the industry's debt conversation on its head. Debt is usually diagnosed by feel (the code smells, the team grumbles, a consultant frowns) and contested by the same means. The audit diagnoses it by citation: *name the answer that forces this position.* Silence is the finding. Chapter 1 promised the thirty-second version: three services sharing a database and deploying in lockstep, where no live answer demands the deployment split and the flag arrives almost before the question finishes. The worked version needs no new system: run it on the enterprise platform Chapter 6 derived. Feed `next_step` that vector and its own answer sheet, unchanged, and every position re-derives to itself — the read-path split still forced by the 200-millisecond contract, event-sourced pricing still forced by the replay mandate, the distributed store still the one container for strict-across-regions. *No change on every axis* is what a healthy system under audit looks like: not fashionable, *cited*. Here the verdict is cheap — the derivation is pages old and nothing has moved. The audit earns its keep on systems whose answers have had years to drift from their structure: run at that scale on a long-

lived payroll platform (the full story is *Process-First Design*'s brownfield chapter), the same re-derivation came back *no change* on every axis, every "except" and special case cited rather than grandfathered.

A word about people, because audit findings land on them. A position flagged as unforced is *not* a decision flagged as wrong: the answers may have moved since it was made — yesterday's forced position is today's unforced one with no error anywhere in the chain — and the audit cannot distinguish a mistake from a world that changed, which is precisely why it should never be asked to assign blame. The vocabulary is load-bearing: positions are "unforced" or "no longer cited," never "wrong"; the decision record's *revisit-when* trigger is the graceful exit built in advance: the original decider named the conditions under which their decision would expire, and the audit merely reports that the conditions arrived. Run this way, the audit is adoptable in organizations where architecture reviews have historically been careers colliding; run as a blame instrument, it will be gamed into uselessness within a quarter, and will deserve it.

The audit's other output is cultural. A system that can answer "why are you shaped this way?" row by row is a system whose architecture discussions have terminating conditions — Chapter 1's promise, extended from design meetings to the standing estate.

One answer moved: incremental recomputation

Answers change one at a time far more often than they change wholesale — a contract lands, a regulator rules, a growth curve bends. The pressure matrix makes the consequence surgical: every position cites the rows that force it, so a changed row implicates exactly the axes it presses and no others. Re-derive those axes; everything else is untouched *by construction*, not by hope.

The same platform carries the worked case. One answer moves: the regulation demanding quote reconstruction under past rule versions is superseded, and no data class now carries a replay demand. The pressure matrix implicates exactly the rows that cited replay — pricing's storage, and nothing else. Re-derive that axis: event-sourced pricing, the most expensive value the whole derivation bought, recedes to current-state with an audit log. The quote path's separation stays — the 200-millisecond contract and the storm shape still force it, and its projections now rebuild from current state — but the versioned event schemas, the replay machinery, and the unbounded log fold away. Everything else — booking, the distributed store, the split — re-derives to itself. One lapsed answer, one moved axis, and — the half the industry finds harder — a *merge* as the computed output. The procedure is direction-neutral: it recommends consolidation with exactly the confidence it recommends splitting, because both are just containment arithmetic. Note how rare that sentence is in this literature. Migration guides run one way; the derivative runs both.

The audit and the increment compose into an operating rhythm: audit on a cadence (or on every planning cycle), recompute on every changed answer. Neither is a project; both are habits with artifacts.

The trail: decision records that write themselves

A derivation's pressure matrix, kept, is the document every inheritor of every system wishes existed: which answers forced which positions, dated. The habit costs one page per axis move — position, forcing rows, mechanism, costs accepted, and the revisit trigger: *this decision expires when these answers change.* That last field converts decision records from archaeology into instrumentation — a standing list of which business facts are load-bearing for which structures, consulted forward rather than excavated backward.

The trail also closes a loop this book opened at the SEI gap. Elicitation feeds derivation; derivation leaves records; records make the *next* elicitation cheaper and the next audit mechanical. The disease the brownfield chapter will confront — a system whose reasons died with its authors — is, for systems built under this method, structurally prevented rather than heroically cured.

What the derivative is not

Two misreadings are worth closing at the source, both because they are natural and because Chapter 7's evidence answers them.

The derivative is a prediction of what organizations will build. It is a computation of what the answers force. Chapter 7's fourth run put the gap on public record: the derivation predicted the forced minimum, the organization had chosen fine-grained decomposition no published demand forced, and the miss was the audit detecting an unforced position through a blind fold — the same test, run without access, still found the uncited structure. Whether an unforced position is debt or an unpublished demand is a question *for the owner*; that the question now has a precise form is the contribution.

The derivative is continuous re-architecture. The regime insight guarantees the opposite: most answer changes press nothing — they land inside containment and the derivative returns "no change" at the cost of a table lookup. Systems under this calculus move *less* than fashion-driven ones, because fashion is a pressure the matrix has no row for.

One case remains, and it is the hard one: several answers moved, or the audit found a vector that was never derived in the first place, and the computed next step stands far from where the system is. Between here and there lie intermediate states, each of which must keep running a business. Getting from a vector to a vector is a path problem, and paths through living systems have physics of their own.

Rules exercised: `next_step` with a non-null starting vector · the audit (position ⇒ citing answer, silence ⇒ debt) · incremental recomputation via the pressure matrix · merge as a first-class output · decision records with expiry triggers · the forced/chosen boundary, applied.

Migration as Pathfinding

> *"Deriving the destination is necessary, but designing the journey from the existing system, with legacy, constraints and real teams, is perhaps where architects truly earn their value."* — Loïc Veyssière, written under the article this book grew from

Two teams inherit the same assignment (the audit has spoken, the monolith's checkout component must become its own deployable with its own store) and they sequence it differently. The first team extracts the component now and plans to separate its data later. The second separates the data first, inside the monolith — per-component ownership of tables, one deployable — and extracts along the clean seam afterward. Same starting vector, same target vector. The first team spends eight months operating a shared database astride a network boundary: cross-component writes that were transactions are now a protocol, while the schema still couples both sides: the costs of two positions, the capabilities of neither. The second team never passes through that state at all.

Deltas do not commute. The order of moves is a real decision with real physics, and it is derivable — from the same instruments, pointed at a different question. The derivation picks the *target*; this chapter picks the *route*: a sequence of steps from the vector you hold to the vector the answers force, where every intermediate state is a system you actually run.

The model

Migration is pathfinding through vector space. **Nodes** are operable vectors: positions per axis, with scopes. **Edges** are deltas: one coherent transformation — an axis move at a scope, a scope split or merge, or a scaffolding step. **Edge weights** are the cost-and-risk indicators, deliberately qualitative: the reversibility of the step; the feasibility of the intermediate state while the system runs through the change; the coupling cost of coordinated changes elsewhere; the duration spent in dual state; the dependency on prior transformations; and the failure-mode amplification an irreversible step carries — which is what earns a step its mitigation plan.

Qualitative weights change how paths are compared: argument, never arithmetic. A path is rejected by *naming the indicator that kills it* — prune-mode reasoning applied to routes — and never by a weighted sum, because a weighted sum over incommensurable risks is taste with decimal places.

Path constraints govern every route, and they are where the opening story lives. Each intermediate state must be **operable**: staffed, debuggable, on-call-able; the intermediate *is* production, usually for months. Each must be **affordable**: dual-running pays the boundary cost from Chapter 3 in duplicate, genuinely if temporarily. And each must be **calendar-feasible**: businesses have immovable walls — filing seasons, year-end closes, retail peaks — and a delta whose dual-state period would span a wall

is infeasible even when the delta itself is small, because the frozen weeks must never contain a half-migrated system. *When* is a path property, and the payroll validation supplied the canonical instance: a one-axis merge, trivial as a delta, that simply cannot transit filing season.

Non-commutativity, worked

The opening pair is the persistence-and-topology case: extract-then-separate transits the infeasible node (the shared store across a network boundary — the distributed monolith, named at last and defined precisely: a topology move whose consistency decayed to protocol while its schema coupling stayed); separate-then-extract keeps every intermediate operable. Same endpoints; the order was the entire decision.

A second shape recurs so often it deserves its own name: **the priced scaffold**. A separated read model needs a change feed. If an event substrate is on the path anyway, sequence it first and the projections ride the published facts: the second delta shrinks. Projections first means building change-capture scaffolding the later substrate move will demolish. That order is *allowed* — sometimes the read path's deadline genuinely comes first — but the scaffold must enter the plan priced as a whole edge: built, operated, and torn down. Pathfinding's contribution is making the throwaway explicit at planning time instead of discovering it as "wasted work" in a retrospective. Chapter 8 called this the scaffolding step from the failure side; here it is a legitimate purchase with a price tag.

And the third shape is triage: deciding what kind of move a "migration" even is, using the vector lens Chapter 7 validated on Discord's two store migrations. A **product swap** moves no axis: same position, different vendor, procurement with operational care, and the documented easy case. A **single-axis delta** at one scope is the ordinary edge. A **compound delta** — several axes at once — is where programs go to die, and the pathfinding response is decomposition: find an order of single-axis deltas whose intermediates are all feasible. When no such order exists, the route needs a scaffolding node, or the target needs Chapter 8's renegotiation menu, because "we cannot get there operably from here" is a contradiction between the target and the calendar, and it prices like any other.

Ordering principles

Dependencies impose a partial order — some deltas are prerequisites for others. Inside that order, one principle does most of the sequencing work, and it is the principle Part I never needed: **irreversibility orders commitment.** Reversible deltas go early — they are experiments the path can retreat from; the hard-to-reverse delta goes as late as the dependencies allow, because an early irreversible step forecloses every alternative route while information is still arriving. Greenfield derivations never fire this rule; every transformation fires it constantly, which is why it lives in this chapter.

Field experience corroborates the rule from an independent direction. Poltorak's catalog (next section) observes, three separate times, in nearly identical words, that certain infrastructure layers, once adopted, are "unlikely to be removed": middleware, shared repositories, proxies are practical one-way doors. The catalog states it as ob-

servation; the indicator set explains it — those layers' removal costs amplify with everything built atop them — and the sequencing consequence is the same either way: treat one-way doors as late-path commitments.

The edge list, and what catalogs price

A pathfinding model wants an edge list: which transitions between positions are known, with what prerequisites and costs. This book's edge list is adapted, with attribution and thanks, from Denys Poltorak's *Architectural Metapatterns* (CC BY 4.0) — whose "Evolutions of Architectures" appendix catalogs seventeen-odd transitions between named shapes, each with prerequisites, benefits, and drawbacks, several with explicit staged decompositions. Revoiced into six-axis vocabulary, his transitions map cleanly onto edges: his prerequisites read as destination-fitness conditions, his drawbacks as always-on costs and failure-mode amplifications.

The mapping also exposed a boundary worth stating precisely, because it defines this chapter's contribution relative to the catalogs. **Transition catalogs price destinations; a migration must price the crossing.** A prerequisite like "the data can be split into semi-independent parts" gates whether the target fits the domain. It says nothing about whether the system stays operable *while the split is in progress*. Intermediate-state feasibility, dual-state duration, calendar walls — the properties that killed the first team's route in the opening — have almost no counterpart in any catalog's fields, and could not: they are properties of *routes*, and catalogs enumerate *places*. Both prices are real. Pay the destination price from the catalogs; derive the crossing price here.

The loop, closed

Each executed delta leaves its decision record — position, forcing answers, costs accepted, revisit trigger — so the path documents itself as it is walked; the inheritor receives the route, and every route survives its droughts better than its destination survives alone. Upstream, the rhythm from Chapter 9 keeps running: the audit detects *that* a step is due, this chapter walks it, and answers changing mid-path re-enter the derivation like answers changing anywhere else: a path is as recomputable as the target it aims at, and long programs that cannot absorb a changed answer mid-route are waterfalls wearing migration branding.

Everything in Part III so far assumed a defensible starting point: a system whose current vector is *known*, whose answers are on file, whose history is legible. The next chapter drops that assumption the way reality does — a system built by people who are gone, for reasons nobody recorded, audited by an institution with no stake in flattering anyone — and runs the entire method against the hardest starting position there is.

Rules exercised: deltas and the six indicators · reject-by-named-indicator · the three path constraints (operable, affordable, calendar-feasible) · non-commutativity · the priced scaffold · migration triage by the vector lens · irreversibility-orders-commitment · destination price vs crossing price.

Brownfield

Most worked examples in architecture books are systems their authors built, which makes every one of them a take-home exam again. This chapter's case is a system nobody reading it built, documented for over a decade by an institution with no stake in the narrative: **Universal Credit**, the United Kingdom's welfare-benefits platform — audited by the National Audit Office in 2013, 2014, 2018, and 2024, examined by Parliament on the record, and carrying the rare thing this chapter needs most: an *admitted* failure, with the reset that followed it documented in public.

It is also, quietly, the book's best evidence for Chapter 9. The system has completed one full derivation cycle — inherited position, reset, rebuilt vector, legacy decommissioned — and a second reform program opened against the rebuilt system while this book was being written. Architecture as a derivative stops being a metaphor here; it is a timeline with an auditor.

One discipline governs everything below, stated once: where the public record attests a fact, it is cited; where the chapter must infer, the inference is labeled **[reconstruction]** and argued from the finding record. A worked example's authority comes from keeping those two registers separate: the same rule the blind derivations ran on, applied to history. (The evidence base is UK public sector three times over in this book, and the reason is structural, not patriotic: the NAO and its parliamentary counterparts *publish the demand side* — budgets, deadlines, failures, write-offs — with a candor no private enterprise volunteers. Gradeable systems live where auditors publish.)

The inheritance: V_0, named honestly

It is 2013, and you have just inherited the Universal Credit "live service." The programme began in 2011: six legacy benefits to be merged into one, delivered under waterfall contracts by four systems integrators: £1.12 billion in contracts let across IBM, Accenture, HP, and BT (contract ceilings of £525m, ~£500m, and £100m for the first three; actual spend to March 2013: Accenture £125m, IBM £75m, HP £49m, BT £16m). By the end of 2012, £638 million had gone to IT, £441 million of it on design and development. In early 2013 a review by Capgemini and an internal reset stocktake concluded the architecture was of "limited extensibility," with — the National Audit Office's own summary — **no detailed blueprint, no target architecture, and no operating model ever produced**. The Major Projects Authority told the Public Accounts Committee that much of the IT was "not fit for purpose" and unlikely to be reusable. Up to £130 million of work was headed for write-off; £34–40 million was formally written down. (The persistent claim that a specific commercial case-management product sat at the core is UNVERIFIED at the primary-source level, and the chapter does not use it; the finding record carries the argument without it.)

Run Chapter 9's audit question against this inheritance — *name the answer that forces each position* — and the audit returns silence on essentially every axis: **V_0 is almost entirely uncited.** Not "bad"; *unforced*, which is the precise finding. The four failure

modes this book's companion methodology names for inherited systems, checked in turn:

Boundaries that predate their reasons. The system's seams were the *contract* seams — four vendors' delivery boundaries, drawn by procurement rather than derived from anything. The record supplies the epitaph, from a consultant on the programme: **"We were effectively on a waterfall project, because it was a waterfall contract."** The contract shape was the architecture shape. No answer was ever asked to force it. This is the rival account of architecture — that contracts and org charts draw the seams, not demands — caught in the act, and naming it is what the method is for: a boundary no answer forces is an unforced position whichever hand drew it, and procurement is only the most expensive hand. The contract did not disprove the derivation. It stood in for one, and the audit trail is the bill.

A model that was never a model. "No blueprint or target architecture ever produced" is this failure mode in an auditor's handwriting. There was nothing to audit positions *against* — the F-question ("what forces this?") had no ledger on either side.

Legacy persistence swallowed the domain. [reconstruction] The volatile part of a benefits platform is the *rules* — welfare policy changes with political weather, and six benefits' worth of live amendment history were being merged. "Limited extensibility" is what that looks like operationally when rules are welded to stored case-record shape: a policy change becomes a schema-and-system change. The inference is argued from the extensibility finding and the contract structure; the product identity it is usually hung on is unnecessary as well as unverified.

The fourth mode — components that must deploy together — is absent, and saying so matters: the failure modes are a checklist, never a prophecy. V_0 failed as a monolith with vendor seams, which is its own way to fail.

The answers, as they stood in 2013

Hindsight is the enemy of a fair reconstruction, so the sheet below admits only commitments on the record *before or at* the reset:

- **Continuity, prune-grade:** existing claimants must be paid, every period, through any transformation. The one non-negotiable — and it survives verbatim into every later cycle of this story.
- **The parliamentary calendar:** national rollout from October 2013, completion by 2017 — politically committed, publicly tracked. In Chapter 2's triage, a *requester's-clock* answer: it binds the programme and will become a path constraint; no latency mechanism can address it.
- **Volume, projected:** seven-million-plus households at full scale — a demand *with a date*, per the entry gate, never a day-one requirement.
- **Fraud and error:** the 2010 White Paper committed to reducing fraud and error by £1 billion a year against a £5.2 billion baseline, and administrative costs by half a billion — a *verification-shaped* demand on the calculation core (correct entitlement, per rule version), which is a consistency answer wearing a fiscal costume.

- **Policy volatility:** [reconstruction, from the regulations record] the Welfare Reform Act 2012 trailed a multi-year series of amending regulations before the first pathfinder claim was taken. The change driver on the rules is permanent and external — the load-bearing answer nobody priced.
- **Digital by default** (White Paper), a security and identity-assurance requirement the 2013 record already shows straining, and a **cost envelope** of roughly £2.4 billion, with a department that had not built software in-house for twenty years — the bound-mode answer that explains the contracts.

The audit, and the auditors

Now derive. Continuity plus policy volatility force **isolation between the paying path and the changing rules** — this period's payments must not sit in the blast radius of next period's rule changes. V_0 welded them: uncontained. The fraud/error commitment forces **entitlement computed reconstructably per rule version** — a replay-shaped demand on one data class [reconstruction at mechanism level]; V_0's current-state case records cannot answer it: uncontained. And the volume projection, decomposed, presses *nothing in 2013*: the pathfinder that eventually launched was deliberately capped at ~1,500 simple claims a month; building day-one for seven million households was the classic unforced position, and the record shows the contracts bought exactly that, at £638 million for a caseload three orders of magnitude smaller.

Read those three findings against the NAO's: "limited extensibility" is the volatility driver uncontained; "no blueprint" is the audit's silence, institutionalized; the write-off is the price of the unforced position. **The auditors ran this book's audit by hand and called it a stocktake.** The derivation adds nothing to their verdict except the part that matters going forward: *why*, row by row, and therefore *what next*.

The reset as a renegotiation menu

2013's position is Chapter 8's structure in the wild: the parliamentary calendar × continuity × an unusable V_0 cannot all hold. The menu, as history actually faced it:

1. **Patch forward** on V_0 — the pre-reset default. Price: the uncontained volatility driver compounds; the record had already priced this option at £130 million of scrap.
2. **Big-bang replace** — stop, rebuild, cut over. Struck on *path* grounds, per Chapter 10: the cutover transits an intermediate state that gambles payment continuity — the prune-grade answer — on an unproven system. This menu entry is how the programme got here; choosing it twice would have been the same derivation error at higher stakes.
3. **Twin-track** — keep the live service paying while building the replacement beside it at the narrowest scope, migrating by readiness. Price: **double-run**, stated in the open — two systems, years of parallel operation, the boundary cost paid in time instead of risk.
4. **Renegotiate the calendar** — the menu's standing fourth entry, because commitments are answers too. The record shows this happened repeatedly: the 2017 target slid, in public, under parliamentary protest.

History chose three *and* four. The claim this chapter makes is deliberately modest: not that anyone ran this procedure, but that **the feasible-path structure was derivable in advance** — and the two years and £34–40 million spent discovering it by exhaustion were the price of not deriving it. The price of the missing *diagnosis*, precisely: whether the organization could have acted on a 2013 derivation inside the contracts it had already signed is a different question, and Part IV names it as one of the method's walls.

The path, walked and graded

The executed route, delta by delta, against Chapter 10's indicators:

1. **Pathfinder at one jobcentre** — Ashton-under-Lyne, 29 April 2013, capped at ~1,500 straightforward single-claimant cases a month, "careful and controlled" in the Secretary of State's own framing. The narrowest-scope delta, fully reversible — close it and the live service still pays — with the scope restriction written into the *eligibility rules*: the demand itself was narrowed, not just the deployment. Reversible-first, textbook placement.
2. **Expansion gated on maturity, not calendar** — test-and-learn, service by service, district by district. Where the calendar reasserted itself the indicators name the cost precisely: the 2018 NAO report records many claimants suffering hardship during the full-service rollout, from design and implementation issues combined — **dual-state duration whose costs were borne by the users**, the one indicator no programme dashboard tracks and the audit record does.
3. **Live service closed to new claims (2018)** — the first hard-to-reverse delta, placed *late*, after the digital service demonstrably carried the full intake load. Irreversibility ordering, executed by instinct.
4. **Live service decommissioned (2019)** — cycle one closes: $V_0 \rightarrow$ reset $\rightarrow V_1$, complete.
5. **Managed migration of remaining legacy claimants** — the statutory-calendar-constrained final delta, still running a decade after the reset; the 2024 NAO report records that one in five invited tax-credit claimants did not complete the move and had benefits stopped. The chapter keeps that sentence. Paths have human costs, auditors record them, and a method that talks about calendar walls owes its readers the number.

The grade on V_1 arrived unrequested, in March 2020. The pandemic put a roughly fivefold demand spike on the rebuilt system — 950,000 applications in a single fortnight, active claims rising toward five million, 2.2 million calls in a day, release cadence forced from six changes to seventy-six urgent changes in two months — absorbed, per the department's own engineering account, without statutory payment deadlines moving. History load-tested the derivative and published the result. Keep the two grades distinct, though, because the record does: the *architecture* absorbed the surge; the *business case's* fraud number did not land (a later NAO assessment found fraud and error above the White Paper's forecast). A derivation answers for the vector. It has never answered for the policy.

Coda: the derivative keeps deriving

In April 2026 the department opened a market notice titled, verbatim, “UC Project Zora — Application Decomposition & Microservices Transition”: a £40 million programme against “the constraints of a long-standing monolithic application estate,” to decompose the rebuilt system progressively “while maintaining continuity of service.” Sixteen years after the White Paper, the continuity answer survives, word for word, into a third cycle’s founding document — and cycle three begins while this book is in print. [One neutral observation the record supports: Zora is itself a procurement seeking an external delivery partner — the 2011 pattern under a different contracting philosophy; whether the new seams follow change drivers this time is a question the audit trail will answer for whoever writes the next edition of this chapter.]

Run the closing question the way Chapter 9 taught: *which answers changed?* Post-pandemic load shape; a decade of accumulated policy deltas; the estate’s own age as an operability answer. The derivative computes the next step; the next step is being taken; the chapter closes on an open case because every real system is one.

What the method has now done — derived, verified, graded blind, halted honestly, audited, walked, inherited — is bounded on every side by things it deliberately does not do. Part IV names them, and names the trend this book joins by refusing to join it loudly.

Rules exercised: the audit against an inherited vector · four failure modes as checklist · hindsight-clean elicitation · requester’s-clock triage at programme scale · the unforced position, priced by an auditor · the renegotiation menu in the historical record · reversible-first and irreversible-late · dual-state costs · the derivative across three cycles.

The Judgment That Stays Human

A book that spends twelve chapters mechanizing decisions owes its last argumentative chapter to the decisions it cannot mechanize — stated precisely, because the credibility of every "derived, not chosen" so far depends on this boundary being real rather than rhetorical. The claim was never that judgment shrinks. The claim is that judgment *relocates*: stripped from the steps that never deserved it, where it was taste with tenure, and concentrated where it is genuinely owed. This chapter is the inventory of where it is owed.

The judgment inventory

Each entry has the same shape: what the mechanics produce, and what a human decides with it.

The answers themselves. The sheet elicits and prices; the business sets every number. The derivation owns *consequences*, never targets — and the pricing discipline cuts both ways: pricing is the method's job, and wanting the expensive thing anyway, eyes open, is legitimately the business's. A priced wish that survives pricing is called a commitment, and the method salutes it.

Recovery ties. Where the domain offers both a defined inverse and tolerable degradation, the ledger prices both and stops. Restored consistency versus continued liveness is a business preference wearing a technical costume — money usually compensates, availability usually degrades forward, and the choice between those characters belongs to whoever owns the outcome. Corollary from the field: when the inverse must be *designed* (an off-cycle correction, a clawback process), compensation is a use case of its own, with its own targets, and deciding it is worth building is a product decision the method can only price.

The renegotiation menu. A contradiction's menu is derived and priced; choosing which commitment bends is the business call *by definition* — the commitments were theirs. Chapter 8's global order book ended with four branches; no procedure picks among them, and one that claimed to would be lying about whose money it is.

The conscious second driver. The method's boundaries fall where pressures diverge, and occasionally a team decides — knowingly — to let one component serve two drivers: a boundary blurred on purpose, for reasons the matrix can record and cannot weigh. Permitted, priced, and *named*: the sin was never the decision; it was the unconsciousness.

Enterprise-altitude inputs. Vision, strategy, and organizational bets enter the sheet as answers — release structure, cost envelopes, multi-X — and the book takes them as inputs honestly rather than pretending to model where they come from. The derivation starts where the business's self-knowledge ends its first draft.

The stance sentence for the whole inventory: **the derivation does not shrink judgment; it removes judgment's counterfeit.** What was defended by seniority now needs no defense, and what genuinely needs deciding now arrives at the decider priced.

The MTTR/MTBF dissolution: a worked example of the boundary

One live industry argument makes the inventory concrete, because this book's instruments dissolve most of it and hand back the remainder — visibly split.

The trend is familiar from every reliability deck of the last decade: *failure is inevitable; stop maximizing time-between-failures and start minimizing time-to-recovery.* The stance descends from the everything-fails school of operations and John Allspaw's influential 2010 essay, whose title contained a scope qualifier, "for most types of failures," that the ensuing decade dropped in transit. The essay's own author named the exception class: failures that must never happen, accidental data loss the canonical case. The slogan erased its own boundary; the recovery axis restores it, *per failure class*: degrade-and-continue is the recover-fast stance formalized; compensate is the there-is-an-inverse stance priced; and **design-out transcends the dichotomy entirely — the failure class stops existing, and both metrics go undefined on it.** Repair-versus-prevent was never one question; it was one question per failure class, and the classes have a derivation.

The metrics half of the trend died separately, and the record deserves its citations: the essay's author retracted the *mean* himself — incidents are not comparable, start and stop times are negotiated attributions; the VOID (Verica Open Incident Database) project confirmed it empirically at corpus scale (duration distributions are heavily skewed; central-tendency measures misrepresent them; duration correlates poorly with severity); and independent statistical work reached the same verdict by Monte Carlo: you cannot reliably demonstrate improvement through a mean of incident durations. This book files the lesson with Chapter 5's: **means lie about incident durations for the same reason they lie about latency — scalars hide scope and shape.** One discipline, both chapters.

What remains standing after the dissolution is instructive, because each piece lands on one side of this chapter's boundary. *Mechanized:* per-operation failure budgets — the error-budget arithmetic that Google's reliability literature states as "target a given availability figure, rather than particular MTBF or MTTR figures," which is Chapter 2's failure budget in an operator's voice, with the two old metrics demoted to derived planning levers underneath a priced target. *Mechanized:* verification by injection — game days and fault drills as the recovery axis's test suite, per Chapter 5. *Human:* which failure classes get which recovery class at what price: the inventory's second entry, arrived at from the field.

The scope boundary

The last honesty is about the book itself, and it has four walls.

Enterprise backend. The hard-real-time tripwire fired at elicitation, in Chapter 2, and this is the chapter that owns the why: a deadline whose breach is a *correctness* failure — the brakes, the pacemaker, the trading circuit-breaker — belongs to a different physics regime with a different literature and a harder bar. Percentile budgets and containment arithmetic are the wrong instruments there, and a method that will not name its regime is selling something.

The socio-technical half. Resilience engineering, the study of how *organizations* anticipate, monitor, respond, and learn, owns the human side of every incident this book's structures merely survive. The recovery axis is the meeting point: this book derives what the *system's structure* offers a failure; the organization's capacity to use that offer is a discipline with its own giants, cited here and not annexed. One import does cross the border with full honors, because it is measurement rather than sociology: availability accounting must match user-visible operations — a fleet at 99.99% behind a broken login is a broken product, whatever the dashboard says. The consistency lens, pointed at metrics.

What the method optimizes for is not what every organization wants. A derivation produces the forced minimum, and Chapter 7's fourth run showed a real organization living deliberately above it. Some of that gap is unpriced option value, some is hiring-market signaling, some is fashion — and sorting a given case into those bins is *owner's judgment about the owner's money*, which the audit can inform and must not usurp.

Acting on the vector. The derivation produces the next correct step; whether an organization can take it is answered by its contracts and its politics, not by its answer sheet. Universal Credit's correct 2013 vector — isolate the paying path from the changing rules — was derivable on the day; acting on it meant redrawing seams that four signed contracts had already fixed, an authority no architecture document confers. The method delivers the diagnosis at the moment it is cheapest to have; the authority to spend it belongs to whoever owns the contracts. A method that promised otherwise would be claiming to derive its way out of a negotiation.

The boundaries drawn, one chapter remains: what to do with all of this on Monday, and what this book asks of its readers in return.

Rules exercised: the judgment inventory · per-class dissolution of a one-bit trend · budgets over means, twice · verification by injection · the four walls of scope.

Closing

The book opened with two teams who never argued because they never met. It closes with the observation that made writing it feel less like proposing something and more like transcribing something: **people keep arriving at these positions from their own starting points.** A skeptic who spent four rounds defending the centralized database ended by proposing service seams inside a monolith with generated remoting: transport transparency, re-derived by an opponent. A theorist working on module cohesion arrived at change-driver partitioning with no shared lineage. A practicing architect read one derivation and immediately located the frontier this book's third part lives on — if the target is computable, the differentiator is the transformation path. None of them was taught the method, and the three are not the same kind of evidence: the opponent and the reader converged in contact with its material, which is what a good argument produces; the theorist arrived with no contact at all, which is what a real convergence produces. The method is, in part, a name for where the field was already converging — the same claim this book's companion makes about design: the job was never to push a standard, only to reveal one forming.

What the pages between added to that convergence is small and, this book hopes, sharp. An answer sheet whose questions had to earn their seats. A ledger where capabilities meet their always-on costs. A procedure that starts at the cheapest position and moves only under citation — with a conflict rule, a contradiction halt, and a refusal list. An exit gate of lens and arithmetic. Derivations against the instruments, by kind: three profiles on home ground, four blind against systems this book had no hand in, one constructed contradiction the mechanics halted on, one national-scale brownfield read against the public record. Fourteen graded positions on the strictest blind run; two misses kept and converted into rules — one the derivation's own, about the boundary between forced and chosen, and one the author's registered taste, which the derivation overrode and beat. A calculus for systems that live: the audit, the increment, the path, the inheritance. And a boundary chapter, because a method that cannot say where it stops is a mood.

Monday

The worksheet in the appendix is the whole method at operating altitude, and the honest way to adopt it is the way every instrument in this book was validated: on something that already exists. Pick the system you know best. Fill Part A — the nine answers, priced, scoped, with UNKNOWN written wherever UNKNOWN is true; the domain-shape facts per effectful operation. Run the audit: for every axis position the system holds, name the row that forces it. You will find inert answers by the handful, at least one position no row forces, and, if the experience of everyone who has run this so far generalizes, one purchase your organization made for a demand that dissolves under decomposition. That half-day costs nothing and tells you whether this book works on your reality. Design-day derivations can wait for the next design day; audits are available immediately.

The standing invitation

A derivation is a claim with its inputs attached — which makes it refutable, and refutability is a feature this book intends to keep. If you run the worksheet on a real system and the method lands somewhere the reality refutes (an axis forced to a value that failed in production, a contradiction the mechanics missed, a demand shape the ledger cannot name), that filled worksheet is the most valuable document this method can receive. The constructed failure case in Chapter 8 holds its place only until a real one replaces it. **A method improves on counterexamples, not on applause** — the four blind derivations were this book stress-testing itself in public, and its readers can apply more stress than its author ever could.

One disclosure belongs in the same breath, because provenance is part of honesty: this method was not designed at a whiteboard and then illustrated. Its mechanics were debugged in the field — most recently against a distributed runtime this book's author builds (the third and final appearance of that disclosure), where a running backlog of deferred decisions, each verified before building and merged only at named stopping points, was the procedure operating live before it had a book. Methods with field scars are the only kind whose failure modes come pre-documented.

The last reframe, once more

An architecture is the next correct step — computed from where the system stands and what the answers now say, recomputed when they change, audited when they don't. The two teams from Chapter 1 were never holding positions; they were holding *outputs*, and the argument between their partisans was never about architecture at all. It was about answers nobody had written down.

Write them down. Derive. When someone disagrees, hand them the sheet — the debate will terminate, one way or the other, and either way the system wins.

The worksheet is Appendix A. The reference cards are Appendix B. The counterexamples are yours to send.

Appendix A: The Derivation Worksheet

The method at operating altitude. Part A is the sheet you fill in; Part B is the reference you consult while filling it. Chapters 2–5 are the full treatment; nothing here is new, and everything here is enough.

Part A — The sheet

Step 1 · Write down the answers

One row per answer. Every answer needs a **number** and a **scope** — "scalability" is not an answer; "the search path serves 50k req/s peak" is.

#	Question	Your answer (number + scope)
1	**Time budget** — per operation: percentiles, tails, soft maxima, completion windows	
2	**Failure budget** — per operation: error budget, criticality	
3	**Loss budget** — per data class: RPO, retention, what may never be lost	
4	**Consistency contract** — per data class or path: strict / bounded / eventual; read-your-writes where?	
5	**Load** — magnitude (steady and peak), shape per path, concentration, window	

#	Question	Your answer (number + scope)
6	**External constraints** — audit vs replay; residency; mandates that strike values	
7	**Release structure** — cadence divergence between parts; deploy safety	
8	**Cost & capacity envelope** — money; who operates	
9	**Multi-X** — regions, tenants, versions: what partitions naturally, what the law pins	

Below the table, list the **domain-shape facts** — one line per effectful operation:

- Does an inverse exist (can it be undone in the domain, not in the database)?
- Does the value decay over time?
- Can the operation be reshaped — made idempotent, commutative, or append-only?

And the **change-driver facts** — one line per rule set or data class: how often does the governing logic change, and under whose control? These rarely press alone; check them in combinations.

An UNKNOWN is a valid answer. Write it down as UNKNOWN — never guess.

Step 2 · Start from the null vector

```
single deployable / direct / unified / current-state / single shared
```

Recovery has no null — it is decided per effectful operation in Step 4.

Step 3 · The pressure pass

Go through the answers one by one. For each, decide: **inert** (the null vector plus a containment rung handles it) or **pressing** (an axis must move). Record only the pressing ones:

Answer (Q#, scope)	Mode	Shape	Axis pressed	Toward	Because (mechanism)

Two rules to keep in view while you fill this:

- A driver presses only when it **crosses what the current vector can contain**. Most answers are inert — that is a result, not a failure.
- Check answer **combinations**, not just rows: some pressures clear their bar only when two answers from different questions converge on the same axis.

Step 4 · Resolve and write the vector

Work through the pressing rows using the resolution rules in Part B, then write the result:

```
topology / substrate / read-write / state / persistence
+ recovery class per effectful operation
```

Every position must cite the answer(s) that forced it. A position no answer forces is debt — either inherited, or about to be built voluntarily.

Step 5 · Verify

- **Per operation:** name the guarantee, the mechanism that earns it, and the behavior under failure. A guarantee with no mechanism means the vector is wrong — or the answer was fake.
- **Arithmetic:** decompose each latency budget down its critical path; compose capacity envelopes upward. If the floors don't fit inside the budget, stop here.

Step 6 · Leave the trail

One decision record per axis move:

> **Position** · forced by (answer #s) · via (mechanism) · costs accepted · **revisit when** (the citing answer changes)

Part B — The reference

Driver modes (what each answer does)

Mode	Questions	What it does	Character
Prune	3, 4, 6, 9	Eliminates values that can't honor it	Binary. Correctness — can't buy back with hardware
Select	1	Chooses among survivors	Presses only near the physics floor

Mode	Questions	What it does	Character
Split	5	Forces decomposition	Presses only on divergence — uniform load presses nothing
Isolate	2	Blast-radius containment	Starts biting past ~99.9 on a named path
Bound	7, 8	The economic envelope	Sets the frame everything optimizes inside

Demand shapes (which mechanism family contains it)

Ask of every load answer: **would a second copy help?**

- **Volume** — yes, a second copy helps. Contained by capacity: sizing, cache, replicas, sharding.
- **Contention** — no: one record, one winner, regardless of copies. Contained by admission control, coalescing, or design-out. Never by sharding.
- **Burst** — a peak with a tolerable settling delay. Contained by a buffer (the queue). If the peak must be served synchronously, it is volume at the peak number.
- **Deadline** — "finish inside the window," not "respond fast." Contained by windowed throughput plus resumable batch. Latency mechanisms are irrelevant.

One more triage for time answers: does the deadline bind the **requester's clock** (a statutory 14-day filing window — inert for architecture) or the **system's clock** (a 200 ms response target — presses)?

Containment rungs (price these before moving any axis)

```
hardware sizing → cache → coalescing → replicas → projections
```

Each rung contains a different pressure; the last one is the axis move, everything below it is not. Thin tiers — load balancers, caches, coalescers — own no business logic and no data of record: they never appear in the vector.

Resolution rules

1. **Rungs before moves.** The cheapest containing value wins; each new mechanism is an ongoing cost, not a one-time price.
2. **Prune-mode demands: try scope exclusion first.** Narrowing where the demand applies (split the regulated path out) competes with hardening an axis — and often wins.
3. **Conflict on one axis → the scope test.** Pressures from *different* scopes: don't compromise the axis — split the system at the boundary between the pressures and derive each part separately (the split itself has a price: a contract, a consistency decay, a translation seam, an operational seam). Pressures from the *same*

scope: decompose the demands to mechanism level and satisfy the binding part minimally.

4. **Still opposed after decomposition → a genuine contradiction.** The output is a renegotiation menu with prices, not an architecture. This is the method working, not failing.
5. **Apply every chosen value at the narrowest scope that contains the demand.** Event-source one data class. Separate one read path. Extract one component. A value applied wider than its demand is unforced cost.

Recovery classes (per effectful operation, from the domain-shape facts)

- **Design-out** — reshape so the failure cannot arise (idempotent, commutative, append-only, structural constraint). Cheapest at runtime; check it first.
- **Compensate (backward)** — a defined domain inverse exists; restore consistency by compensating. The inverse must stay in-domain, or compensation becomes its own use case.
- **Degrade-and-continue (forward)** — defaults, queue-for-later, decay states, checkpointed batch. Degraded windows must be bounded and visible.

Appendix B: Reference Cards

One card per instrument. Definitions live in Part I; these are for the desk.

Card 1 — The nine questions

1. **Time budget** — per operation: percentiles, tails, soft maxima, windows. *Hard max as correctness = out of scope (hard real-time).*
2. **Failure budget** — per operation: error budget + criticality. 99.5% ≈ 44 h/yr · 99.9% ≈ 8.8 h/yr · 99.99% ≈ 53 min/yr.
3. **Loss budget** — per data class: RPO, retention, never-lose set.
4. **Consistency contract** — per data class/path: strict / bounded (named bound) / eventual; read-your-writes where?
5. **Load** — magnitude (steady/peak), shape **per path**, concentration, window.
6. **External constraints** — audit (*who/what/when*) vs replay (*state under past rules*); residency pins; mandates that strike values.
7. **Release structure** — cadence **divergence**; deploy safety. The count presses nothing; divergence presses everything.
8. **Cost & capacity envelope** — money + *who operates*.
9. **Multi-X** — countries/currencies/tenants/regions/versions: partition gifts + legal pins.

Second row source: domain-shape facts per effectful operation — inverse exists? value decays? reshapeable (idempotent / commutative / append-only)? · change-driver facts per rule set / data class — how often does the governing logic change, under whose control? (PFD's change-driver analysis feeds here; presses in combination, rarely alone.)

Entry gate: priced (a nine has a price; "what changes in the 53rd minute?") · scoped (per operation / data class / path) · decomposed ("team independence" → ownership ≠ release independence; "audit" ≠ replay) · triaged (requester's clock vs system's clock; observed failure ≠ stated target) · contradictions surfaced early.

Card 2 — The six axes

Axis	Values (null first)
Deployment topology	**single deployable** (± modules; ± role-selective activation) / multiple / unified runtime / serverless
Composition substrate	**direct** / event-based / streaming
Read/write model	**unified** (+ chain rungs) / separated
State storage	**current-state** (+ audit log) / event-sourced
Persistence	**single shared** / distributed shared / sharded (→ cells) / per-component / polyglot

Axis	Values (null first)
Recovery *(no null)*	design-out / compensate (BER) / degrade-and-continue (FER)

Values apply **at demand scope**. Hybrids are compositions: parts + boundary cost. Deployment prices two countables: release coupling + delivery plumbing (pipelines, the version matrix) bill to the *artifact*; scaling shape and runtime blast bill to the *unit*. Unique container: strict × multi-region × zero-loss on one data class → **distributed shared** only.

Card 3 — Demand shapes

Would a second copy help?

- **Volume** (yes) → sizing, cache, replicas, sharding.
- **Contention** (no — one record, one winner) → admission control (write side), coalescing (read side), design-out. Never sharding.
- **Burst** (peak + tolerable settling delay) → buffer/queue. Synchronous peak = volume at the peak number.
- **Deadline** (finish inside window) → windowed throughput + resumable batch. Latency mechanisms irrelevant.

Time-answer triage: **requester's clock** (statutory window — inert) vs **system's clock** (response target — presses).

Card 4 — Containment rungs

```
hardware sizing → cache → coalescing → replicas → projections
                                                  ↑ the axis move
```

Thin tiers (LB, cache, coalescer, admission gate) own no business logic and no data of record → never in the vector. Hardware rung's ceiling is *economic*: it ends where the working set or write rate outgrows one box's price curve.

Card 5 — The procedure (`next_step`)

0. **Null vector**: single deployable / direct / unified / current-state / single shared. (Living system: start from its actual vector.)
1. **Normalize** — run the entry gate.
2. **Prune** — correctness answers strike values (binary).
3. **Press** — per demand: contained → *inert* (a result); uncontained → record (axis, direction, scope, mechanism). **Check combinations, not just rows.**
4. **Resolve** — cheapest containing value · fewest new mechanisms · narrowest scope · rungs before moves · scope exclusion before hardening. Recovery per effectful operation from domain shape (check design-out first).
5. **Verify** — the exit gate (Card 6).

Conflict rule: opposing pressures on one axis → *different scopes*: split at the boundary (pay: contract, consistency decay, translation seam, ops seam) · *same scope*: decompose further · *still opposed*: **contradiction** → renegotiation menu with prices (bend the softest demand first; every branch re-enters the derivation).

Refusals: targets, recovery ties, contradiction choices, product picks.

Card 6 — Verification

Consistency lens, per operation: **guarantee + mechanism + failure behavior**. Guarantee without mechanism = wrong vector or fake answer. One-bit labels banned in the system's own docs.

Budget arithmetic: 1. Latency decomposes down the critical path — sequential adds, parallel costs its max, hops pay floors. Target ÷ floor → 1 = pressing; floors > target = wrong vector. 2. Tails compose through the distribution — slow-fractions add in series (5 steps at P99 each ⇒ ~5% chain-slow); fan-out harvests the tail (100-way at per-shard P99 ⇒ ~63% of requests see one). 3. Envelopes compose upward **by correlation** — steady adds; peaks add only when correlated; the calendar is capacity input. 4. Availability multiplies in series (five 99.99% parts ≈ 99.95%); parallel arithmetic requires **earned independence** (no shared deploys/certs/regions/config). 5. Mechanism bill fits the operating envelope — count the vector's standing mechanisms (stores, brokers, projection pipelines, cell disciplines); price the count in Q8's two currencies (money + operating attention). A comparison rather than a formula: a platform-team vector against a four-engineers sheet fails before anything is built.

Pre-build: load models (with correlation), capacity vs rung ceilings, failure injection per lens row.

Card 7 — The living system

Audit — for every held position: *name the answer that forces it.* Silence = debt, by construction. Unchanged answers must derive "no change."

Increment — a changed answer implicates exactly the axes its rows press; re-derive those; the rest is untouched by construction. Merges are outputs too.

Path — deltas don't commute. Constraints per intermediate: operable · affordable · calendar-feasible (dual state must not span a freeze wall). Weigh by six indicators (reversibility, intermediate feasibility, coupling, dual-state duration, dependency, failure amplification) — reject by *named indicator*, never by weighted sum. Reversible deltas early; one-way doors late. Scaffolds are allowed when priced with their own demolition.

Trail — per axis move: position · forcing answers · mechanism · costs accepted · **revisit when**.

References

Works engaged with, grouped by the role they play. Evidence sources for the blind derivations and the brownfield case are listed by system, since their citations anchor specific graded claims.

The shelf this book sits on

- Martin Kleppmann, *Designing Data-Intensive Applications* (O'Reilly, 2017) — the physics this book's ledger presupposes; also the source of the discipline against one-bit consistency labels.
- Mark Richards & Neal Ford, *Fundamentals of Software Architecture* (O'Reilly, 2020) — the modern catalog, engaged as the genre's best exemplar.
- Denys Poltorak, *Architectural Metapatterns: The Pattern Language of Software Architecture*, v1.2 (2026), metapatterns.io, CC BY 4.0 — the transitions catalog Chapter 10's edge list is revoiced from, with attribution and gratitude.
- Juval Löwy, *Righting Software* (Addison-Wesley, 2019) — volatility-based decomposition, the nearest ancestor of change-driver reasoning.
- Gregor Hohpe, *The Software Architect Elevator* (O'Reilly, 2020) — the first-derivative framing of the architect's role.
- Sergiy Yevtushenko, *Process-First Design* (Leanpub) — the companion volume: the design methodology whose architecture-synthesis module this book supersedes and extends.
- SEI: Quality Attribute Workshop and ATAM method documentation (Software Engineering Institute, CMU) — the mechanized front and back doors of Chapter 1's middle room.

Reliability lineage (Chapter 12)

- John Allspaw, "MTTR is more important than MTBF (for most types of F)" (2010), and his subsequent retraction of the mean as a metric.
- The VOID (Verica Open Incident Database) reports and Courtney Nash's analyses — the empirical case against central-tendency incident metrics.
- Štěpán Davidovič, *Incident Metrics in SRE* (O'Reilly, 2021) — the Monte Carlo confirmation.
- Google CRE, "Available… or not?" (Google Cloud blog) — error budgets as priced availability answers; MTBF/MTTR as derived planning levers.
- Werner Vogels' "everything fails all the time" operational stance; GameDay / resilience-engineering literature (Robbins, Krishnan, Allspaw; Hollnagel's four cornerstones) — the socio-technical boundary this book cites and does not annex.

Evidence: Stack Overflow (Chapter 7)

- Nick Craver, "Stack Overflow: The Architecture — 2016 Edition"; "What it takes to run Stack Overflow" (2013); "Stack Overflow: How We Do Deployment — 2016 Edition" (nickcraver.com).

- High Scalability: “StackOverflow Update: 560M Pageviews a Month, 25 Servers” (2014); “StackExchange’s Performance Dashboard”.

Evidence: Shopify (Chapter 7)

- Shopify Engineering: “E-Commerce at Scale: Inside Shopify’s Tech Stack”; “A Pods Architecture to Allow Shopify to Scale”; “Surviving Flashes of High-Write Traffic Using Scriptable Load Balancers” (Parts I–II); “Deconstructing the Monolith”; “How Shopify Reduced Storefront Response Times”; BFCM 2021 engineering retrospective; “Capacity Planning at Scale”.
- InfoQ: “Shopify’s Architecture to Handle Flash Sales” (conference presentation).

Evidence: Discord (Chapter 7)

- Discord Engineering: “How Discord Stores Billions of Messages” (2017); “How Discord Stores Trillions of Messages” (2023); “How Discord Scaled Elixir to 5,000,000 Concurrent Users”; “Why Discord is Switching from Go to Rust”; “How Discord Supercharges Network Disks for Extreme Low Latency”.
- ScyllaDB tech talk: “How Discord Migrated Trillions of Messages from Cassandra to ScyllaDB”.

Evidence: Companies House (Chapter 7)

- Companies House annual reports and accounts (2023–24, 2024–25); the ECCTA 2023 outline transition plan (gov.uk); Companies Act 2006 (legislation.gov.uk), notably ss. 853A, 1084, 1096–1097 and Part 35; the Registrar (Annotation, Removal and Disclosure Restrictions) Regulations 2024.
- companieshouse GitHub organization — service documentation cited in grading; companieshouse.blog.gov.uk engineering posts (2019) — the projection pipeline and platform self-description.
- GOV.UK guidance: second filings (RP04), registrar’s rules and powers, personal information on the public register.

Replication artifacts (Chapter 7)

- github.com/siy/derivation-artifacts — the replication kit: registered predictions, answer sheets, quarantine rules, grading rubrics, the fourth run’s derivation transcript and operator prompts verbatim, and the answer-sheet schema in summary form (the full specification tracks the engine work). The four pre-repository runs are attested (their registered-before-graded ordering is documented internally, not externally timestamped); every subsequent run is registered prospectively, by commit, before outcomes are checked. Counterexamples are invited there as issues: a filled sheet whose derivation diverges from a system you know.

Evidence: Universal Credit (Chapter 11)

- National Audit Office: "Universal Credit: Early Progress" (2013, HC 621); "Rolling Out Universal Credit" (2018); "Progress in Implementing Universal Credit" (2024).
- "Universal Credit: welfare that works" (White Paper, Cm 7957, 2010) and its launch statements; Welfare Reform Act 2012 and Commencement Order No. 9 (the 29 April 2013 pathfinder).
- Computer Weekly's Universal Credit reporting (2013–2016) — contract structure, write-offs, the waterfall-contract testimony; Public Accounts Committee oral evidence (2013–2024).
- DWP Digital, "DWP's agile response to COVID-19: scaling Universal Credit to meet demand" (December 2020) — the surge record.
- "UC Project Zora — Application Decomposition & Microservices Transition" (PIN notice, April 2026); PublicTechnology coverage (June 2026).

A note on one absent citation

Yannick Loth's self-published papers on the Independent Variation Principle and cohesion are engaged in the acknowledgments rather than cited as corroborating evidence: his theory is still under active development by his own account, and this book cites as evidence only what its evidence chapters can grade. The convergence on change-driver partitioning is acknowledged as convergence; readers interested in the relation can read both bodies of work and judge.

Revision History

[1.0.1] — 2026-07-24

Terminology pass, shared with the companion editions.

Changed

- **Numbers no longer brand the label-only sets**: the driver modes, the cost-and-risk indicators, and the path constraints are introduced by name rather than by count. The counts that carry an argument are kept — the nine questions (an output of the membership criterion) and the six axes (with their completeness argument).
- **Recovery value name standardized**: the axis table's *design-the-failure-out* becomes *design-out*, matching the ledger and the rest of the series.

[1.0.0] — 2026-07-19

First edition. Content as of 0.3.15 (fresh-read pass, user read feedback items 1–4, external review 2 disposed — records in book-arch-meta/), plus:

Fixed

- **Ch. 7 and References: replication-kit claims brought current with the live repository** (github.com/siy/derivation-artifacts). Ch. 7's disclosure said the kit "is being prepared" while the references already cited it; both now state exactly what is published — answer sheets, registered predictions, grading rubrics, the fourth run's derivation transcript and operator prompts verbatim, schema in summary form — with prospective-by-commit registration from the repository's creation onward, and the four pre-repository runs attested.

[0.3.15] — 2026-07-19

Added

- **Ch. 4: "What the procedure is, mathematically"** — new section naming `next_step` as constrained optimization over a finite design space, with its three boundaries stated: the space is chosen, not complete (new mechanisms enter as ledger amendments; minimality is guaranteed within the ledger run against, not over all possible engineering); the objective is not a scalar (currencies don't convert, mechanism count is the honest comparator, residual ties go to the refusals by design); the minimum is global over the space (the order argument carries the weight). Closes with why "derivation" still earns its name: the constraints are elicited facts. Answers external review 2 items 1/5/7/8/9.

- **Ch. 3: containment defined** as a ledger-discipline bullet — a claim about a bound, exactly as probabilistic as the bound it serves, inheriting the bound's assumptions, surviving or dying at the exit gate's arithmetic (review item 4).
- **Ch. 3: recovery axis asymmetry owned** in the entry — five axes position structure, recovery prescribes behavior under failure; seat earned under the same membership criterion; "nothing requires the dimensions to be the same kind of thing" (review item 6).

Changed

- **Ch. 3: axes empiricism stated** — count-is-output mirrored from Ch. 2 into the membership-criterion paragraph ("the criterion is the theory, six is its current result"); no completeness proof claimed (review item 2; reviewer's suggested PFD pointer dropped — the referenced PFD discussion does not exist).
- **Ch. 3: ledger schema articulated** — the provides/mechanism/costs fields named as the joints the procedure articulates; requires/excludes columns declined in-text (exclusion is the absence of a provides; a prerequisite is a mechanism) (review item 3).
- **Ch. 1: press/inert marked as terms of art** at selection rule 2 (review editorial); site glossary gains both entries.

Fixed

- **Ch. 3: distributed-shared-store uniqueness claim scoped to the ledger** ("the only value *in this ledger* that provides…") — review's correctness item; the sibling passage in Ch. 6 already used the scoped form. Site glossary entry aligned.

Review record: `book-arch-meta/architecture-synthesis-review.md` (verbatim), `REVIEW-2-DISPOSITION.md` (per-item rulings incl. rejections with evidence and the post-1.0 deferred register: worked tie, verify-correct-rederive loop, diagrams).

[0.3.14] — 2026-07-19

Changed

- **Ch. 5 budget arithmetic restructured into math → example pairs** (user proposal during the read). Each rule now carries its law sentence, one set-off formula line in Unicode notation (no LaTeX math mode — portable to every downstream format), and the worked example + discussion. Rules 1/2/4 get full formulas (path-floor subtraction; series slow-fraction addition + p/n allowance + fan-out $1 - (1 - p)^n$; series/parallel availability products with plugged numbers 0.9999^5 and $1 - 0.001^2$); rule 3 gets its composition rule-form; rule 5 gets its inequality (Σ mechanism bills ≤ operating envelope) and keeps its "comparison rather than a formula" character. Rule 1's worked example is now the chapter's opening quote contract itself — the ninety-second arithmetic the cold open says nobody ran (200 − 80…150 geography = 50–120 ms for all software), closing the loop instead of gesturing at it.

[0.3.13] — 2026-07-19

Fixed

- **Ch. 4's closing hook no longer pre-states Ch. 5's thesis** (user read finding, same ruling as 0.3.12: reader-experienced repetition wins). The chapter close and Verification's opening both said "a derived vector is a set of claims — this guarantee, from this mechanism, within this budget" plus the wrong-vectors/fake-answers function, one page apart. The definitional home is Verification, whose two checks the triplet maps onto; Ch. 4's close is now a lean hand-off — every position is a claim, claims get checked, the last instrument does the checking.

[0.3.12] — 2026-07-17

Fixed

- **Ch. 4 Step 0 cites the null vector by name** instead of re-enumerating its five values (user read finding; anti-repetition rule 3 — the null vector's home is Ch. 3's selection rule, the reference cards carry the verbatim list). Reverses the 0.3.1 repetition-pass ruling that had kept the enumeration as the procedure chapter's own artifact: the reader experiences it as repetition, and the reader wins. Step 0 keeps what is new at its site — recovery has no null, the living-system starting position, and the measurement-precondition defense.

[0.3.11] — 2026-07-15

Changed

- **Ch. 3, deployment-topology entry: the delivery plumbing priced** (user finding during the read). The artifact's bill extends beyond release coupling: pipelines, release processes, and dependency streams multiply per artifact, and independent cadences buy a **version matrix** — production runs a mixture of versions through every rollout window, so the tested composition is no longer the only one running and cross-artifact compatibility becomes a standing bill (contract tests, deprecation windows, N-by-M pairs). Priced in both directions: the single artifact's train grows with the codebase (one test gate every team's merge waits behind), contained a long way by module-aware builds and affected-only test selection, and capping out as a *cadence* demand — pressing the axis through the answer sheet rather than around it. The single deployable's provides-line gains "one composition in production — the composition you test is the composition that runs."
- **Card 2** billing note echo: delivery plumbing bills to the artifact.

[0.3.10] — 2026-07-15

Changed

- **Series note:** supersession sentence updated for the convergence landing — PFD 2.1.0 asks the same nine questions; earlier editions' eleven noted as historical, transition story in PFD's changelog.

[0.3.9] — 2026-07-15

Changed

- **Ch. 2: the eleven→nine audit narrative moved out** (user read-feedback item 1). The chapter keeps the membership criterion, its provenance in one clause, and the count-is-output law; the historical arc (which questions merged and why; the PFD-reader migration note) relocates to PFD's revision history — specified in `book-pfd-meta/PLANNED-CHANGES.md` item 12; both books converge to nine at PFD's next revision; the published articles stay at eleven (historical). Also resolves the fresh-read "evidentiary IOU" on the unshown audit claim.
- **Series note** now carries the cross-book note itself (pre-convergence PFD editions ask eleven where this book asks nine; PFD's revision history has the transition) — replacing the pointer to a ch. 2 migration note that no longer exists.
- **Ch. 1** promise updated to match: the next chapter holds questions to a membership criterion (no longer "watch that audit happen").

Fixed (queued fresh-read findings, folded into the same pass)

- **Nine reads as nine:** the bound-mode questions split into their two entries (Release structure · The cost and capacity envelope) — the in-prose tally now matches the count.
- **Shape vocabulary glossed at first mention** (volume / burst / contention / deadline, one clause each); the ledger keeps the full treatment.
- **Change-driver facts:** explicit reassurance that the sheet does not require the companion book — the question as stated produces the row; PFD owns the systematic method.

[0.3.8] — 2026-07-15

Changed

- **Ch. 3, deployment-topology entry rewritten around the artifact/unit distinction** (user finding during the read: role-selective activation — one artifact, handlers/process classes enabled per instance group — defeats the old "whole-unit scaling" cost line). Costs now bill to their countable: release coupling → artifact; scaling shape + runtime blast → unit; network-in-crossing → boundaries the composition actually crosses (largely pre-paid on an event substrate). The role-selective form named with its record (web/worker processes, database node roles) and its honest limits (one release train; runtime disable = kill-switch, not cadence; dormant-handler wake-up shares the envelope). Unified runtime re-

grounded: the role-selective form is its hand-rolled ancestor; the platform class is young, the pattern is not (shrinks the disclosure's exposure). Multiple deployables narrow to what only artifact multiplicity buys. Validation record checked: no graded derivation flips (every derived split also cites cadence, polyglot, or blast; Profile 3's cores pick is now better-cited — direct-call coupling is what forces the productized form).
- **Card 2** synced: role-selective activation listed; artifact/unit billing note added.

[0.3.7] — 2026-07-14

Added

- **Ch. 3:** the three-spaces model named and defined (demand / selection / position spaces) — closes the spine + instruments-tag promise; **team topology** run against the axis-membership criterion as the second shown rejection (Conway's-Law candidate → existing axes + unforced positions).
- **Ch. 4:** order-independence paragraph — Step 4 stated as computing Ch. 3's global cheapest-containing vector; the mechanism count is global over the finished vector.
- **Ch. 7:** methods-note honesty extensions — parametric-memory limit named (procedural vs informational blindness; the topology miss as contamination counter-evidence); target-selection vs execution-blinding distinction; pre-committed disconfirmations for the topology rule ("provably" dropped from prose and grade table).
- **Ch. 9:** in-book worked audit + increment on the Chapter 6 enterprise profile (replay mandate lapses → event-sourced pricing recedes; the quote path's separation stays, its citations intact); payroll case reweighted as imported at-scale evidence, callback-only.
- **Ch. 12:** fourth scope wall — acting on the vector (derivability ≠ authority to act; Universal Credit as the exhibit).

Changed

- **Ch. 1:** founding move sharpened — rival drivers (team structure, contracts, risk appetite) named and folded in as inputs-or-debt before "there is a better account."
- **Ch. 11:** waterfall-contract epitaph extended into the rival-account exhibit (the contract stood in for a derivation; the audit trail is the bill).
- **Ch. 6:** author-graded concession moved to the chapter door (exit line becomes a callback); unified-runtime pick annotated (sole selection of the disclosed value in the book; never demanded by the blind runs).
- **Ch. 7:** "in both directions" dropped (both realized cases resolved the same way); scorecard splits the misses by kind (author's prior vs derivation's own); sharper-baseline concession added (an architect's registered judgment is the real rival — exists once in this set, the replication kit invites the corpus); rung-zero culture tied back to Ch. 3's unnamed mention.
- **Ch. 8:** bend-A2 branch names the scope discipline (a bend applies at the narrowest relieving scope; one market leaves none to cut).

- **Closing:** tally restated by evidentiary kind; the two misses split; convergence anecdote graded by evidence kind (contact vs independent); gap-drain disclosure de-jargoned.
- **Ch. 3:** "mechanism-counting" explicitly stated as the precise sense of Ch. 1's "arithmetic" (refinement, not retreat); quorum commit glossed at first use.

Fixed

- **Ch. 5:** 99.5% correctly labeled two nines and a half (was "three nines and a half").
- **Ch. 8:** cross-region RTT aligned to Ch. 5's 80–150 ms (was 70–140).
- **Spine:** derivation.md blurb no longer promises "ordering principles" (that section belongs to Ch. 10); now says what the chapter states (why axis order does not matter); brownfield blurb reframed retrospective ("read retrospectively against," not "graded by").
- **Ch. 12:** VOID spelled out at first use; Allspaw named in-body.

Verification round (skeptic re-read of the changed passages: 12 closed, 5 improved)

- **Ch. 1:** recovery axis flags its input class at first use (domain-shape facts, the second input class Ch. 2 formalizes); "prices them as debt" softened to "a price" (the debt judgment lives in Ch. 9, with its falsifier).
- **Ch. 9:** pre-committed disconfirmation for "unforced = debt" (option value exercised → missing question, sheet grows) — the same medicine the topology rule got; closes the cross-cutting unfalsifiability the verify pass named.
- **Ch. 11:** the £34–40M counterfactual scoped to the missing *diagnosis*, with an explicit pointer to Part IV's acting-on-the-vector wall (the two chapters no longer pass in silence).
- **Ch. 7:** Discord's C-grade re-flagged at point of use in the cross-findings.

(Source: three-persona voice-blind fresh read, reconciled in `../book-arch-meta/FRESH-READ-SYNTHESIS.md`; edits per `../book-arch-meta/PROPOSED-EDITS.md`, forks A1=C, A4=C, three-spaces=demand/selection/position, RTT=80–150 user-ruled 2026-07-14.)

[0.3.6] — 2026-07-13

Changed

- **References, Loth note simplified at his request** ("stay on the safe side, don't overstate until my work is more stable"): the proofs-audit clause removed; the stated reason is now his own still-under-development framing; readers invited to judge the relation themselves. Nothing promised on either side.

[0.3.5] — 2026-07-12

Added

- **Ch. 10 epigraph:** Loïc Veyssière's comment, used with permission (granted 2026-07-12).

[0.3.4] — 2026-07-12

Added

- **References:** the replication-kit repository cited (github.com/siy/derivation-artifacts), with the attested-vs-prospective registration distinction stated — completes review item 4.1's "cite in References" now that the repository exists.

[0.3.3] — 2026-07-12

Changed

- **Real cover replaces the placeholder** ("the vector" concept): six axes, candidate values as open ticks, the derived vector as one bold polyline — in the series visual language (black line-art on warm off-white, scattered shapes, one spot of colour). The amber dot is the leftmost position: PFD's output is this book's origin (user ruling on series-token semantics). Concepts A ("unfolding point") and C ("pathfinding lattice") retained in the meta dir as the design record.

[0.3.2] — 2026-07-12

Fixed

- **Sync echoes dropped by 0.3.0's staging:** Card 6 gains Rule 5's echo (mechanism bill vs operating envelope); Card 1 and Appendix A gain the change-driver-facts second-row extension (the disposition's 3.1 "Touches" items for the appendices).

[0.3.1] — 2026-07-12

Fixed

- **Ch. 11:** NAO 2018 hardship attribution corrected to the source's wording — design and implementation issues combined, not rollout pace (dual-state-duration reading stays as the chapter's own inference). Sourcing debt closed: NAO 2018 quotes, identity-assurance wording, and the £2.4bn reference verified verbatim against primary sources (corpus updated); amendment-frequency confirmed correctly [reconstruction]-labeled (no primary verbatim exists).
- **Repetition pass (voice anti-repetition rule 4):** "an SLO you haven't priced is a wish" re-landing in ch. 2 compressed to attribution; ch. 6 audit/replay definitional restatement compressed to verdicts-plus-name. Ch. 4's Step-0 null-vector enumeration ruled legitimate (mechanics-as-artifacts; procedure's home chapter).

Changed

- **Em-dash budget pass (voice §5)** across all 13 chapters: 547 → 412, punctuation-only conversions (parentheticals → commas/parentheses, clause-joins → colons/periods); structural leads, artifact notation, and quoted material exempt; word streams verified unchanged.

[0.3.0] — 2026-07-12

Added

- **Series front matter** (`series-note.md`): the layers pipeline, reading map, register note, PFD-module supersession note, no-warranty disclaimer.
- **Ch. 1:** synthesis disanalogy (formal inputs vs elicited answers; adoption-curve honesty); fourth obligation (naming where judgment remains); claims legend (derived / empirical / heuristic / contextual); assumptions statement.
- **Ch. 2:** PFD named as the eleven-question predecessor + migration note; the audit/replay/ **erasure** three-way decomposition; second row source extended with **change-driver facts** (the PFD seam — closes the volatility-input gap the series review found).
- **Ch. 3:** recovery-triple unification (long names canonical prose, BER/FER official short forms — series ruling); erasure containment mechanisms (scope exclusion, crypto-shredding, tombstoning) + the statutory replay-vs-erasure contradiction; security topology shown failing the axis-membership criterion; AI-era demand-shape note; rung-zero economics metal-vs-cloud.
- **Ch. 4:** decomposition termination fixpoint (mechanism level is the floor).
- **Ch. 5:** Rule 5 (mechanism bill vs operating envelope — bound-mode verification closes); correlation caveat extended to Rule 2.
- **Ch. 7:** evidence grades A/B/C with per-run labels; attested-vs-verifiable registration disclosure + replication-kit commitment; survivorship-selection note; null-model baseline ("always predict the boring monolith") with the wins over it enumerated; **isolated operators disclosed as AI agents, foregrounded as mechanizability evidence**.
- **Ch. 8:** fifth halt restored (unexplored territory — the instrument-gap halt); statutory contradiction + ch. 11 field-specimen bracket around the constructed case.
- **Ch. 9:** socializing audit findings ("unforced," never "wrong"; blame-free by construction; revisit triggers as graceful exits).

Changed

- "Phase-5/Phase-6" vocabulary excised (standalone-reader fix); reference cards carry the BER/FER short forms; build spine includes the series note.

[0.2.0] — 2026-07-11

Added

- **Full first draft, all chapters.** Part 0 (Two Teams, with the miniature derivation), Part I (Answer Sheet, Axes & Ledger, Derivation, Verification), Part II (Three Profiles, Systems Nobody Asked Us to Derive, When the Derivation Says No), Part III (Derivative, Pathfinding, Brownfield/Universal Credit), Part IV (Judgment, Closing), acknowledgments, Appendix A (worksheet), Appendix B (reference cards), references.
- Voice: per the shared base + arch overlay (hybrid discipline, anti-repetition rules, positive-formulation rule, mechanics-as-artifacts, evidence register, rules-exercised tags).

[0.1.0] — 2026-07-11

Added

- Book scaffold at writing-go: reading-order spine (`root.md`), 17 chapter/appendix stubs matching the confirmed 13-chapter plan (BOOK-PLAN decisions A–C, G resolved), build script (`../book-arch-meta/build-pdf.sh`, adapted from the PFD pattern, no-despine), placeholder cover.
- All chapter material assembled and validated pre-scaffold: ledger v0.2, worksheet v0.2, four blind derivations (10/2/2 of 14 on Companies House under the isolated-operator protocol), Universal Credit brownfield case with verified corpus, 17-edge Metapatterns transition list (CC BY 4.0).